A Project Report On

INTELLIQUIZ —A GAMIFIED ADAPTIVE QUIZ GENERATOR USING LLMs WITH REAL-TIME ANALYTICS

Major Project Phase – 01 submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY
(2022-2026)
BY

GNANESHWAR REDDY YENNA	**22241A1264**
PONNAMANDA ESWAR	**22241A1244**
MADUPATHI NITHIN KUMAR	**22241A1233**

Under the Esteemed Guidance
of
MRS. A. PAVITHRA
Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY
GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)
HYDERABAD

2025-26

CERTIFICATE

This is to certify that the project report titled **"INTELLIQUIZ — A Gamified Adaptive Quiz Generator using LLMs with Real-Time Analytics"** is a bonafide record of work carried out by **GNANESHWAR REDDY YENNA (22241A1264), PONNAMANDA ESWAR (22241A1244), and MADUPATHI NITHIN KUMAR (22241A1233)** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY** of Gokaraju Rangaraju Institute of Engineering and Technology, Hyderabad, during the academic year 2025–26.

MRS. A. PAVITHRA
Assistant Professor
(Internal Guide)

Dr. Y. J. Nagendra Kumar
Head of the Department

(Project External)

ACKNOWLEDGEMENT

We take immense pleasure in expressing gratitude to our Project guide, **Mrs. A. Pavithra**, Assistant Professor, Dept of IT, GRIET. We express our sincere thanks for her encouragement, suggestions and support, which provided the impetus and paved the way for the successful completion of the project work.

We wish to express our gratitude to **Dr. Y. J. Nagendra Kumar, Head of the Department**, our Project Coordinators **Mrs. T.N.P. Madhuri** for their constant support during the project.

We express our sincere thanks to Dr. Jandhyala N Murthy, Director, GRIET, and Dr. J. Praveen, Principal, GRIET, for providing us the conductive environment for carrying through our academic schedules and project with ease.

We also take this opportunity to convey our sincere thanks to the teaching and non-teaching staff of GRIET College, Hyderabad.

Email:gnaneshwarreddyyenna@gmail.com
Contact No: 9398414053

Email: ponnamandaeswar1978@gmail.com
Contact No: 7569833318

Email: madupathinithinkumar@gmail.com
Contact No:9014679452

DECLARATION

This is to certify that the mini-project entitled "**INTELLIQUIZ — A Gamified Adaptive Quiz Generator using LLMs with Real-Time Analytics"** is a bonafide work done by us in partial fulfillment of the requirements for the award of the degree **BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY** from Gokaraju Rangaraju Institute of Engineering and Technology, Hyderabad.

We also declare that this project is a result of our own effort and has not been copied or imitated from any source. Citations from any websites, books and paper publications are mentioned in the Bibliography.

This work was not submitted earlier at any other University or Institute for the award of any degree.

GNANESHWAR REDDY YENNA	**22241A1264**
PONNAMANDA ESWAR	**22241A1244**
MADUPATHI NITHIN KUMAR	**22241A1233**

ABSTRACT

In modern education, static assessments fail to adapt to learner differences and do not scale well in real-time classroom environments. **IntelliQuiz** is an AI-powered, gamified quiz platform that addresses these gaps by generating high-quality multiple-choice questions (MCQs) from diverse learning resources (PDFs, videos, raw text) and adapting question difficulty in real time based on student performance. The system integrates a Java Spring Boot backend with a React + Tailwind frontend, uses PostgreSQL for persistence, and secures access via JWT-based role management for Admin, Teacher, and Student roles.

IntelliQuiz leverages Retrieval-Augmented Generation (RAG) and large language models (LLMs) to produce multi-level MCQs aligned to course outcomes. An Adaptive Difficulty Engine dynamically adjusts question selection using a student model (based on past attempts, accuracy, and time per question), ensuring an optimal challenge point that promotes learning. Gamification elements — points, badges, and leaderboards — increase engagement, while real-time analytics dashboards provide teachers and students with actionable insights on performance, topic weaknesses, and learning trajectories.

The platform emphasizes scalability, modularity, and fairness: LLM outputs are validated with automated quality checks, and analytics enable instructors to audit item difficulty and discrimination. Designed for deployment on common cloud platforms or locally, IntelliQuiz reduces teacher workload by automating quiz creation, grading, and feedback while maintaining academic integrity through item verification and monitoring tools. This project demonstrates how combining LLMs, adaptive algorithms, and gamification can make assessment dynamic, personalized, and pedagogically sound.

Keywords: Adaptive Learning, Gamification, Retrieval-Augmented Generation, Large Language Models, Real-Time Analytics, Role-Based Access, React, Spring Boot, PostgreSQL.

Domain: Artificial Intelligence in Education **(AIED)**, with subdomains in **Adaptive Learning, Educational Analytics, and Gamified e-Learning Systems**.

TABLE OF CONTENTS

LIST OF DIAGRAMS

1. INTRODUCTION

1.1 Introduction to Project

In today's digital learning era, traditional quiz systems fail to provide personalized learning experiences or real-time adaptability. Most assessments are static — offering identical difficulty levels to all learners, without considering their prior performance or learning pace. Teachers often spend hours manually designing quizzes, grading responses, and analyzing outcomes, which limits their ability to focus on personalized instruction and student engagement.

To address these challenges, **IntelliQuiz** introduces an intelligent, **AI-powered adaptive quiz generator** that leverages **Large Language Models (LLMs)** such as Google Gemini and OpenAI GPT to automatically generate **multi-level MCQs** from uploaded materials like PDFs, YouTube transcripts, or textual notes. The system dynamically adjusts quiz difficulty based on each student's performance using an **Adaptive Difficulty Engine**.

Furthermore, **gamification** elements such as **points, badges, and leaderboards** are integrated to boost student motivation and participation. The platform supports **three primary roles** — Admin, Teacher, and Student — each with distinct functionalities and access privileges. Teachers can create quizzes, assign them to classes, and view performance analytics, while students receive personalized feedback and progress tracking in real time. Admins oversee the platform's operations, ensuring integrity and smooth functioning.

By combining **LLM-driven automation**, **gamified engagement**, and **real-time analytics**, IntelliQuiz revolutionizes assessment into a more dynamic, efficient, and personalized experience for learners and educators alike.

1.2 Problem Statement:

The Existing quiz systems are static, repetitive, and non-adaptive. They neither adjust to student learning curves nor provide meaningful feedback that fosters improvement. Teachers face significant time and effort constraints when manually creating question banks, verifying difficulty levels, and analyzing student performance data.

Moreover, students often find assessments monotonous and lack motivation due to the absence of engaging features such as rewards or challenges. The current systems also fail to offer **data-driven insights** — teachers have limited visibility into student learning patterns, while students lack personalized recommendations for improvement.

Therefore, there is a need for a **smart, adaptive, and gamified quiz platform** that can automatically generate domain-specific quizzes using AI, dynamically tailor difficulty to each student, and provide both **instant grading** and **real-time performance analytics** for teachers and learners.

1.3 Objective:

The main objectives of IntelliQuiz are:

1) **Automated Quiz Generation** – Use LLMs to generate high-quality MCQs from varied educational content such as PDFs, videos, and text files.
2) **Adaptive Difficulty Control** – Implement algorithms that adjust question difficulty in real time based on a learner's accuracy and progress.
3) **Gamified Learning** – Increase student engagement through points, badges, leaderboards, and challenge modes.
4) **Real-Time Analytics** – Provide teachers and students with dashboards that visualize performance trends, topic mastery, and learning gaps.
5) **Scalable Role-Based System** – Enable role-based dashboards for Admins, Teachers, and Students with secure access and intuitive interfaces.
6) **Cloud-Ready Architecture** – Design a scalable and easily deployable solution using open-source technologies for academic and institutional use.

1.4 Scope:

1.4.1 Academic Use

The IntelliQuiz platform can be deployed across universities, schools, and online learning portals. Teachers can upload subject materials, generate quizzes automatically, and monitor class-level analytics. Students can participate in gamified quizzes, receive adaptive questions, and track their performance through visual dashboards.

1.4.2 Scaling

Built on a modular microservice architecture (Spring Boot backend, React frontend), IntelliQuiz can scale seamlessly for institutions with thousands of concurrent users. Cloud-based deployment ensures reliability, while containerization (Docker) and database optimization enable efficient performance.

1.4.3 Extensibility

The system is designed with interchangeable modules — quiz generation, analytics, gamification, and authentication — which can be enhanced or replaced without affecting the overall system. New quiz types, feedback modes, or LLM models can be integrated with minimal configuration.

1.5 Technologies Used

(1) **Core Language:**

Java 17: The primary backend language, used to build the Spring Boot-based REST APIs for authentication, quiz management, and analytics.

JavaScript (React): Used for building dynamic and responsive frontend interfaces.

SQL: Used for querying and managing structured quiz and analytics data within PostgreSQL.

(2) **Web Framework & UI:**

Spring Boot: Provides a robust backend framework with built-in dependency injection, REST API management, and integration with PostgreSQL.

React.js: Used for developing a responsive, modular, and component-based frontend.

Tailwind CSS: Ensures modern, responsive, and consistent styling across all user dashboards (Admin, Teacher, Student).

Recharts: A React-based charting library used for displaying analytics and performance trends.

(3) Data Handling & Storage:

PostgreSQL: Serves as the primary relational database for storing users, quizzes, scores, and performance analytics.

JPA (Hibernate): Used as the ORM layer for object-relational mapping between Java entities and database tables.

(4) Machine Learning & AI Integration:

Google Gemini Pro / OpenAI GPT API: Used for automatic question generation, quiz difficulty tagging, and AI-based feedback.

Retrieval-Augmented Generation (RAG): Enhances quiz quality by retrieving relevant context before LLM-based question generation.

LLM Evaluation Module: Integrates AI-based question ranking, validation, and adaptive scoring based on student responses.

(5) Security & Authentication:

Spring Security + JWT (JSON Web Token): Provides secure, stateless authentication with role-based access control for Admin, Teacher, and Student.

BCrypt: Ensures secure password hashing and authentication validation.

(6) Analytics & Visualization:

Recharts: For visual dashboards and charts (performance tracking, topic mastery, leaderboard rankings).

Spring Data JPA Queries: Used for fetching analytical data and aggregating student scores and quiz statistics.

(7) Utilities & Build Tools:

Maven: Dependency management and build automation for the backend.

Docker: Containerization for scalable deployment across environments.

Postman: API testing and documentation.

GitHub: Version control and team collaboration.

(8) Development & Environment Management:

VS Code: For frontend React development.

IntelliJ IDEA: For backend Spring Boot development and debugging.

Environment Configuration: `.env` and `application.yml/application.properties` files store API keys, database credentials, and LLM configurations securely.

2. LITERATURE SURVEY

As we explore the state of the art in **AI-driven adaptive learning and quiz generation**, this survey reviews eight recent studies that integrate **Large Language Models (LLMs)**, **Retrieval-Augmented Generation (RAG)**, and **gamified adaptive systems**. These works collectively highlight the evolution of intelligent educational systems and the foundation for how **IntelliQuiz** bridges existing research gaps through a unified architecture that combines adaptive learning, gamification, and analytics.

1. Chaudhary et al. (2025) presented an AI-based Intelligent Integrated Knowledge Discovery Platform that leverages NLP models such as BERT and GPT-4 for automated question generation and adaptive difficulty adjustment. Reinforcement learning is used to align generated questions with learner proficiency levels. While the system achieves high question quality, it requires large datasets and high compute power, making real-time scalability difficult in typical academic settings.

2. Singaravel et al. (2025) proposed a Smart MCQ Generator that uses Retrieval-Augmented Generation (RAG) and advanced prompting methods like chain-of-thought reasoning and self-refinement. This system automatically generates contextually relevant MCQs tailored to students' prior responses. However, the study highlights the need for integrated analytics and gamification to maintain long-term engagement, which IntelliQuiz addresses through adaptive dashboards and leaderboard mechanics.

3. Sreekanth et al. (2025) research introduced UC-100 Agentic AI Quiz Generation, a multi-agent AI tutoring system that uses LangChain, FAISS embeddings, and Google Gemini 1.5 LLM to generate personalized quizzes. The system achieved over 93% question accuracy using contextual retrieval and self-verification. Its multi-agent workflow inspired IntelliQuiz's modular architecture for content retrieval, question generation, validation, and analytics layers.

4. Maity & Deroy (2024) conducted a large-scale study titled The Future of Learning in the Age of Generative AI, emphasizing how LLMs can autonomously generate assessments and provide adaptive feedback using zero-shot and chain-of-thought prompting. Although effective, they note that maintaining question fairness and controlling bias are ongoing challenges. IntelliQuiz integrates automated question-quality checks to mitigate such issues and ensure academic integrity.

5. Strielkowski et al. (2025) *AI-Driven Adaptive Learning for Sustainable Educational Transformation*, analyzed over 3,000 publications and found exponential growth in adaptive learning systems post-COVID. The research concluded that AI-personalized learning increases accessibility and learning outcomes, but emphasized the importance of data ethics and user trust. IntelliQuiz adopts these principles by implementing secure authentication, transparent analytics, and privacy-preserving data handling.

6. Gómez Niño et al. (2024) *Gamifying Learning with AI*, systematically reviewed how AI-powered gamification enhances engagement and fosters 21st-century skills such as critical thinking and creativity. The authors proved that game elements like points and leaderboards significantly increase motivation and participation. IntelliQuiz extends these principles by embedding gamified progression and reward mechanics in adaptive assessments.

7. Weiß (2025) *Role of Gamification and Adaptive Learning in Engaging 21st-Century Students*, Weiß discusses industry trends showing that integrating gamification with adaptive AI systems can personalize instruct ion efficiently. The study reinforces IntelliQuiz's design approach, combining adaptive difficulty and gamified user experiences to sustain engagement and inclusivity.

8. Wang et al. (2023) AI-Based Quiz System for Personalized Learning (iQS) project integrated AI-generated quizzes into Moodle LMS. Using ontology-driven knowledge modeling, it tailored quizzes and feedback to individual learners. The system validated that AI-driven personalization improves retention and engagement, directly supporting IntelliQuiz's objective of automated, adaptive, and scalable quiz-based learning.

1. Findings of Literature Survey:

After reviewing these studies, several critical findings emerged regarding the current state of AI-based learning and assessment systems:

1. **Automation with Contextual Understanding**

   Most systems successfully generate quiz content using NLP or LLMs but lack mechanisms for continuous contextual adaptation or performance feedback loops.

2. **Adaptivity vs. Scalability Trade-Off**

   While adaptive algorithms improve personalization, many frameworks struggle to scale efficiently due to dependency on large training datasets or compute-intensive LLMs.

3. **Gamification Integration**

   Few systems incorporate game elements effectively into adaptive quizzes. Those that do often lack analytics and sustained engagement mechanisms.

4. **Analytics and Explainability Gaps**

   Many tools generate quizzes but fail to provide interpretive analytics for teachers and personalized insights for students, reducing pedagogical value.

2. Gaps Identified:

a) **Lack of Unified Adaptive Gamified Platforms**:

   Most systems specialize in either adaptive learning or gamification but rarely integrate both in a cohesive, analytics-driven manner.

b) **Absence of Real-Time Analytics**:

   Existing tools lack robust dashboards that visualize student progress, topic mastery, and class-level trends in real time.

c) **Manual Effort in Quiz Creation**:

   Educators still spend substantial time curating, tagging, and validating question banks, limiting scalability and personalization.

d) **Limited Personalization Depth:**

   Many existing quiz generators use static or rule-based difficulty adjustments rather than data-driven adaptivity based on learner behavior.

e) **Insufficient Feedback Mechanisms:**

   Human evaluators may introduce subjectivity, resulting in inconsistent scoring Few systems provide actionable, AI-generated feedback or post-assessment recommendations for learners.

3. Proposed Solution:

IntelliQuiz synthesizes four major innovations into a single adaptive learning pipeline:

1. **LLM-Powered Quiz Generation**
   Utilizes **Google Gemini / OpenAI GPT APIs** to automatically generate, validate, and tag multi-level MCQs from user-uploaded learning resources.
2. **Adaptive Difficulty Engine**
   Implements real-time difficulty adjustment algorithms that personalize question selection based on student accuracy and performance trends.
3. **Gamified Learning Framework**
   Integrates **points, badges, leaderboards, and challenge modes** to boost student motivation and maintain engagement through competition and rewards.
4. **Real-Time Analytics and Feedback Dashboard**
   Provides instructors and learners with dynamic visual dashboards (via **React + Recharts**) that show topic mastery, class averages, and performance improvement areas.

3. SYSTEM ANALYSIS

3.1 Functional Requirements:

1. User Management & Authentication

The system should allow three types of users — **Admin**, **Teacher**, and **Student** — each with distinct privileges.

JWT-based login and registration functionalities must be implemented.

Role-based access control should determine access to dashboards and system actions.

2. Content Ingestion & Processing

Teachers should be able to upload resources (PDFs, text files, or YouTube transcripts).

The system should preprocess the content to extract textual information for AI processing.

3. AI-Based Question Generation

The system should utilize **Google Gemini Pro** or **OpenAI GPT models** to automatically generate MCQs, each tagged with difficulty levels (Easy, Medium, Hard).

Generated questions must include correct answers and distractors for each option.

Questions should be stored in a centralized PostgreSQL database, linked to course topics and learning objectives.

4. Adaptive Difficulty Control

The platform must adjust question difficulty dynamically based on each student's performance (accuracy, time, and previous attempts).

The **Adaptive Engine** should ensure each student receives a customized quiz experience that promotes optimal challenge levels.

5. Gamification & Engagement

Students should earn **points, badges, and ranks** based on performance and consistency.

A **leaderboard** should be maintained at both classroom and institution levels.

Teachers should have control over gamification settings (points distribution, reward thresholds, etc.).

6. Real-Time Analytics & Feedback

The system must provide an analytics dashboard for both students and teachers.

Teachers should be able to view performance trends by student, topic, or class.

Students should receive personalized feedback highlighting strengths, weaknesses, and suggested improvement areas.

7. Quiz Management

Teachers should be able to create, schedule, and assign quizzes to classes or individuals.

Students can attempt quizzes within the allotted time.

Admins can view overall usage, analytics reports, and system activity logs.

8. Reporting & Data Export

The system must generate **PDF or CSV reports** for performance summaries, quiz statistics, and leaderboard rankings.

Admins and teachers can export analytics data for academic review

3.2 Non-Functional Requirements:

1.Performance

The system should generate and deliver quizzes within 3–5 seconds of user interaction.

Real-time analytics dashboards should update with less than 2 seconds of latency.

2. Scalability

The system should handle up to 5,000 concurrent users without significant degradation in performance.

Support horizontal scaling through Dockerized microservices.

3. Reliability

The system should maintain 99.5% uptime during active academic hours.

Automatic retries and failover mechanisms should ensure quiz continuity in case of LLM API downtime.

4. Security

All user credentials must be securely encrypted using BCrypt.

Use JWT authentication for stateless session management.

Implement Role-Based Access Control (RBAC) to restrict unauthorized access.

Use HTTPS/TLS for all data transmissions.

5. Maintainability

The codebase should follow modular design principles for ease of updates.

Each module (QuizService, UserService, AnalyticsService, etc.) should be independently deployable.

Continuous Integration/Continuous Deployment (CI/CD) pipelines should be implemented

via GitHub Actions or Jenkins.

6. Usability

The UI should be intuitive, responsive, and mobile-friendly using React + Tailwind.

Teacher and student dashboards should provide clear visual feedback and navigation.

7. Data Integrity

PostgreSQL transactions should ensure ACID compliance.

Backup jobs should run automatically every 24 hours to ensure recoverability.

8. Compatibility

Compatible with major browsers (Chrome, Firefox, Edge, Safari).

Backend deployable on any platform supporting Java 17+.

Frontend responsive on desktops, tablets, and smartphones.

3.3 Feasibility Study

1. **Technical Feasibility**

   The IntelliQuiz system is **technically feasible** as it leverages proven technologies like **Spring Boot**, **React**, and **PostgreSQL**, combined with scalable APIs for **Google Gemini** and **OpenAI GPT**. The use of **Docker containers** ensures smooth deployment across platforms. The adaptive engine is computationally efficient and runs on standard institutional infrastructure.

2. **Economic Feasibility**

   All components used (Spring Boot, React, PostgreSQL, Docker) are **open-source**, making the solution cost-effective. The only optional expense is API usage for LLMs, which can be managed through cost controls and caching of generated questions. Maintenance and scaling costs remain minimal, especially under hybrid or local deployments.

3. **Operational Feasibility**

   The platform fits naturally within academic workflows. Teachers can upload course content, generate quizzes automatically, and view dashboards with minimal training. Students can log in, take adaptive quizzes, and track progress instantly. With an intuitive interface and automated quiz workflows, IntelliQuiz significantly reduces manual workload for educators.

4. **Overall Risk Assessment**

   The IntelliQuiz system anticipates several operational and technical risks and incorporates measures to mitigate them effectively. In the event of **LLM API downtime**, the platform

employs **local caching mechanisms** to ensure quiz generation and evaluation processes continue seamlessly without dependency on external APIs. To address **data privacy risks**, the system enforces robust encryption protocols for sensitive user data and implements **role-based access control (RBAC)** to restrict unauthorized access. The issue of **system load** during peak usage is managed through **autoscaling** and **Docker orchestration**, enabling dynamic resource allocation and maintaining stable performance under high concurrency. Finally, to encourage **instructor adoption**, the system is designed with an intuitive user interface and guided onboarding workflows, minimizing the learning curve and ensuring smooth integration into existing academic processes.

4. SYSTEM DESIGN

CONCEPT OF UML:

Unified Modeling Language (UML) is a mature language for visualizing software systems and components. It is a way of visually illustrating the components of a complex system and the relationships between its components through a set of symbols (diagrams). UML can be used to show, describe, and share any system's structure and description of work performed by software developers. UML is helpful for the detailed design of large complex software systems while providing a clear and well-understood vernacular between the developers, analysts, and stakeholders. As a design tool, UML has its merits since it easily accommodates object-oriented solutions and follows some acceptable practices for best practices in software engineering;
UML is a productive means of planning, structuring, and organizing the life of software projects.

4.1 - UML DIAGRAMS:

UML is a model-oriented, platform neutral (can be used with any programming language and any development methodology) modeling language, and because it is a standard many software developers will be familiar with it. In other words, although many engineers may not like to use diagrams, in an Agile development environment they add value little by little while also facilitating the pace of work. It will be useful if you consider UML diagrams not as something to just do and an extra nice artifact, but part of documentation.

UML diagrams can help engineering teams in many ways:

a. To welcome a new member onto the team or an experienced developer who is new to the team at the company.
b. How to conduct their activities for source code.
c. This includes thinking about what new features are part of a project before programming begins.
d. Facilitating an easier way to understand information where there may be an interaction among the technical and non-technical observers.

Diagrams can be categorized in UML…

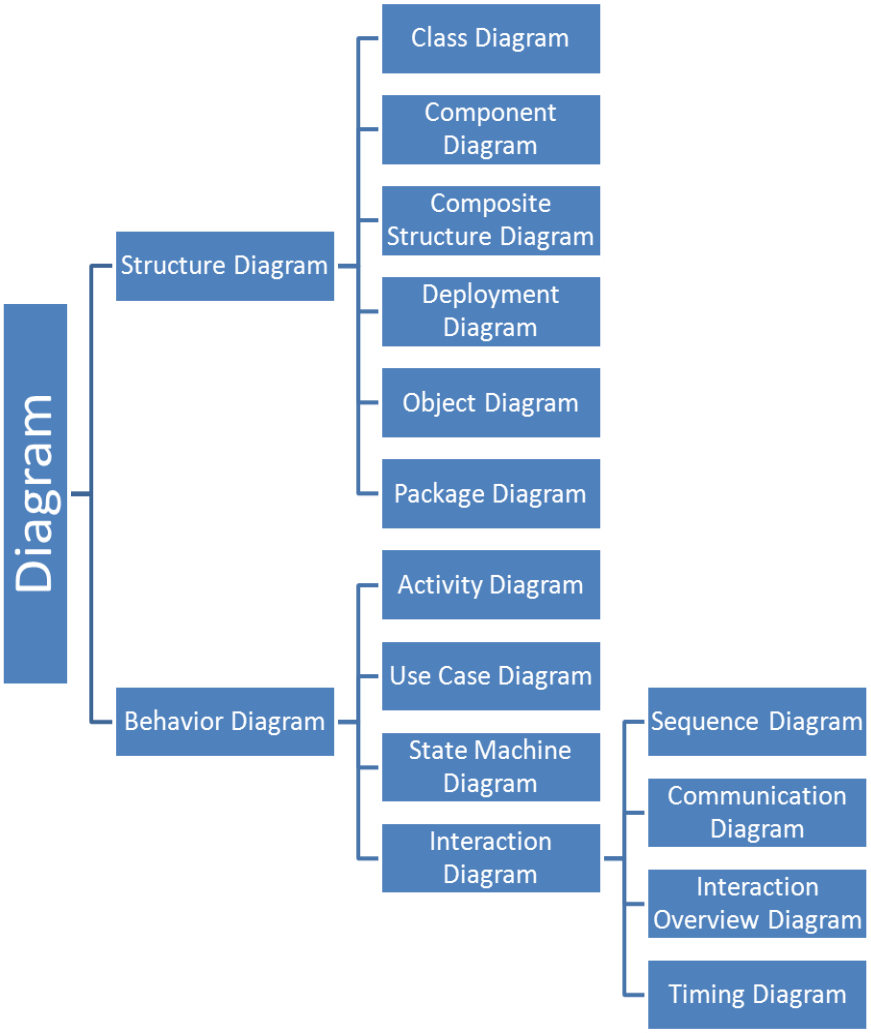

Fig -1: Concepts of UML

4.2 Use case Diagram:

The **Use Case Diagram** represents the interactions between users (actors) and the system's functional modules. IntelliQuiz supports three main roles: **Admin**, **Teacher**, and **Student**.

- The **Admin** manages user accounts, oversees quiz operations, and monitors system performance.
- The **Teacher** uploads learning resources, generates quizzes via the LLM module, assigns quizzes to classes, and views analytics reports.
- The **Student** attempts quizzes, earns points and badges, and reviews performance feedback through interactive dashboards.

This diagram captures the high-level functionalities such as *Login/Register*, *Generate Quiz*, *Attempt Quiz*, *View Results*, *View Leaderboard*, and *Manage Users*, illustrating how each role interacts with the core modules of IntelliQuiz.

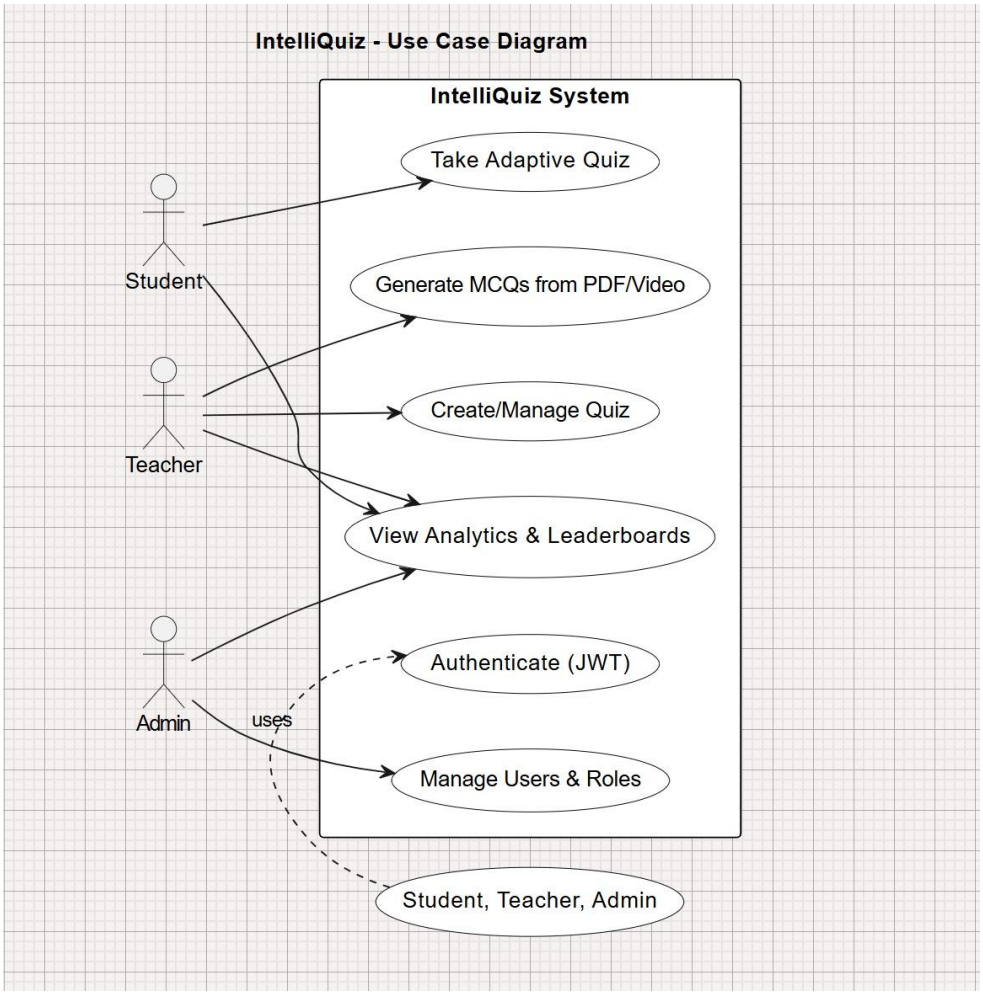

Fig -2: Use Case Diagram

4.3 Class Diagram:

The **Class Diagram** depicts the static structure of IntelliQuiz, showcasing the major classes, their attributes, operations, and relationships.

Key classes include:

- **User** (attributes: userId, name, email, role, password)
- **Quiz** (attributes: quizId, topic, difficultyLevel, createdBy, timestamp)
- **Question** (attributes: questionId, text, options, correctAnswer, difficultyTag, topicTag)
- **StudentAttempt** (attributes: attemptId, userId, quizId, score, accuracy, timeTaken)
- **Gamification** (attributes: userId, points, badges, rank)

- **Analytics** (attributes: userId, quizId, topicPerformance, progressChart)

Relationships:

- *User* → *Quiz* (One-to-Many: a teacher can create multiple quizzes)
- *Quiz* → *Question* (One-to-Many: a quiz contains several questions)
- *User* → *StudentAttempt* (One-to-Many: each student may attempt multiple quizzes)
- *Gamification* and *Analytics* are associated with *User* entities for performance tracking and visualization.

This diagram defines the logical structure and interdependencies among IntelliQuiz's core classes.

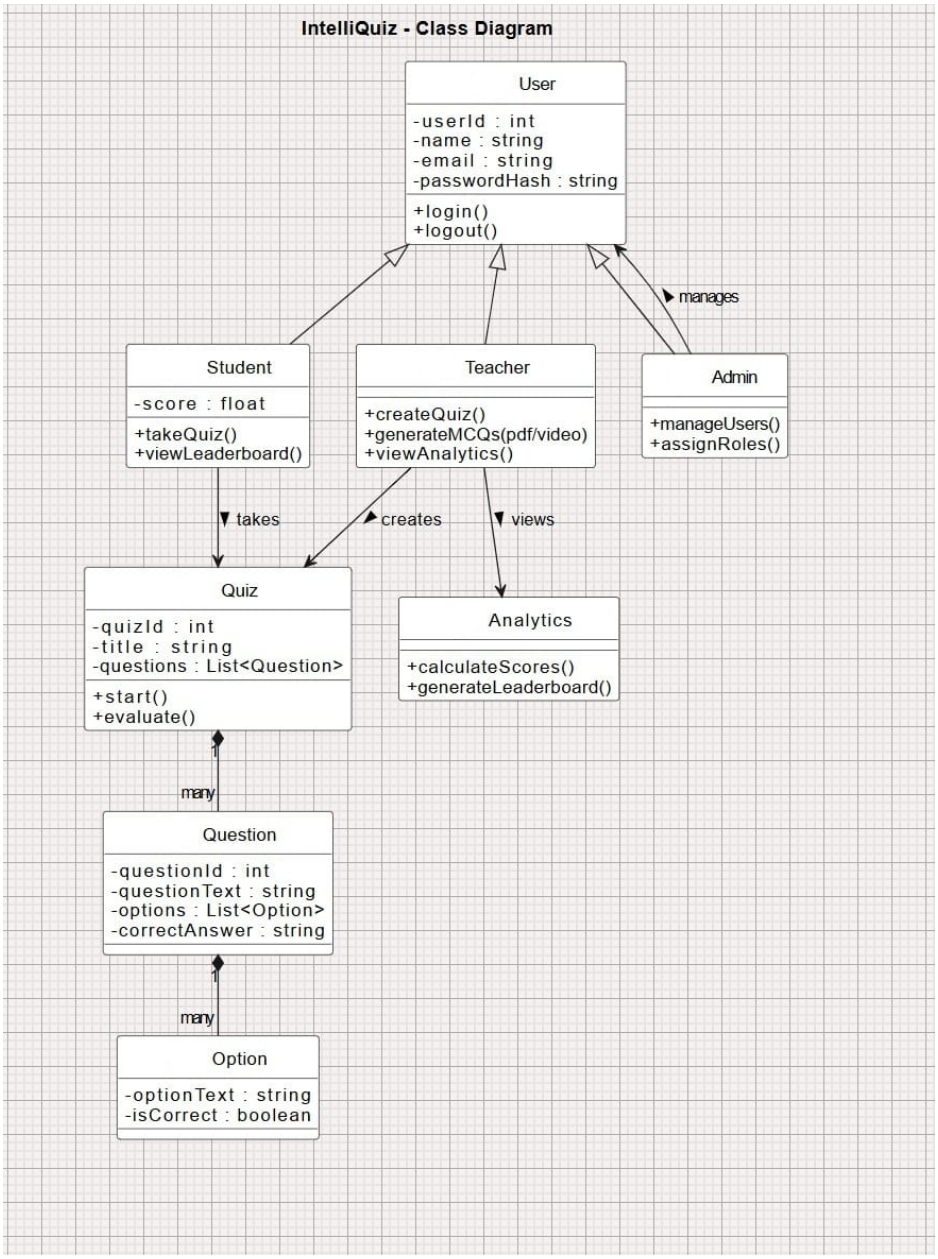

Fig-3: Class Diagram

4.4 Sequence Diagram:

The **Sequence Diagram** illustrates the dynamic flow of operations in IntelliQuiz — specifically how different components interact over time during a quiz session.

1. The **Teacher** uploads content (e.g., PDF or text) via the frontend interface.
2. The **Backend (Spring Boot)** triggers the **LLM Integration Service**, which processes the input using the **Gemini/OpenAI API** and generates tagged MCQs.
3. The generated questions are stored in the **PostgreSQL Database**.
4. When a **Student** initiates a quiz attempt, the **Adaptive Engine** selects questions based on prior performance.
5. Upon completion, the system stores results, updates gamification points, and refreshes the analytics dashboard.
6. The **Teacher** and **Student** can then retrieve analytics reports through the **React Dashboard** in real time.

This diagram demonstrates the step-by-step message flow between the **User Interface**, **Backend Services**, **LLM API**, and **Database**.

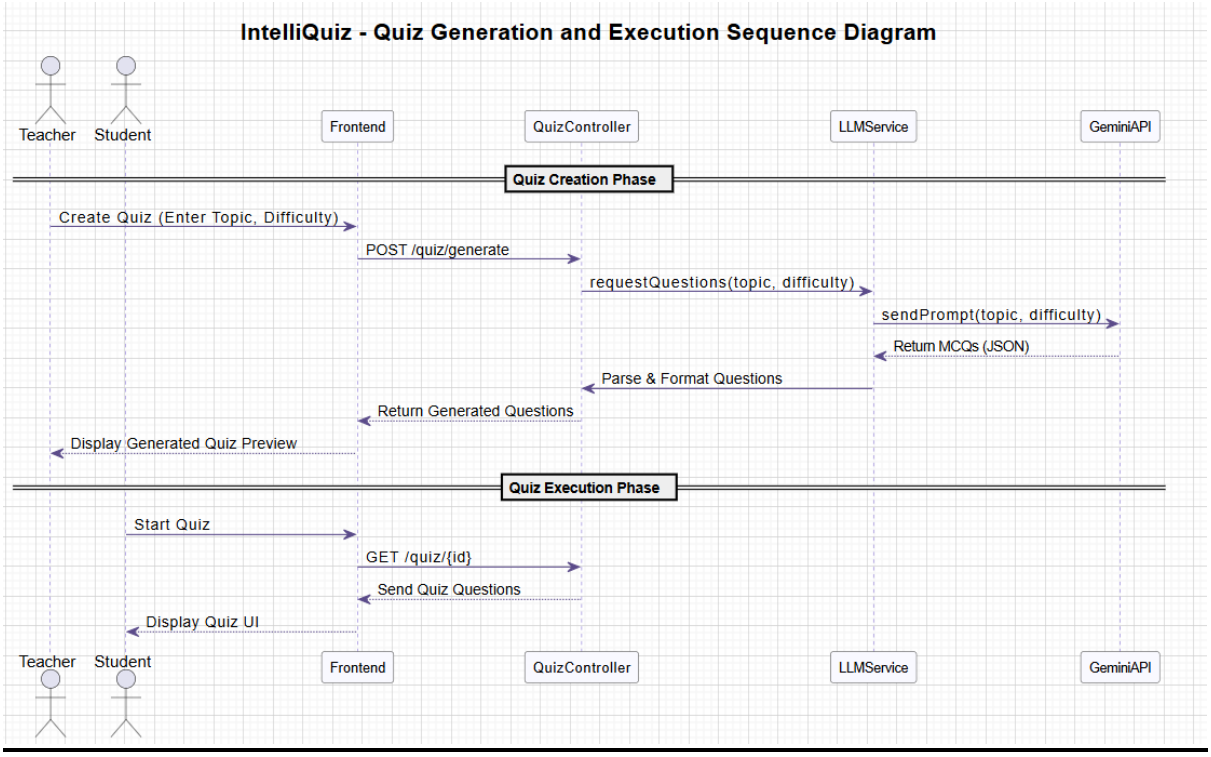

Fig-4: Sequence Diagram

4.5 Deployment Diagram:

The **Deployment Diagram** illustrates the physical configuration of IntelliQuiz and the interaction between software and hardware nodes in the deployment environment.

The platform consists of three primary nodes:

- **Client Node (Frontend):** Hosts the React.js interface accessed via browsers by Admin, Teacher, and Student users.
- **Application Server Node (Backend):** Hosts the Spring Boot application, including modules for Authentication, Quiz Generation, Adaptive Engine, Gamification, and Analytics.
- **Database Server Node:** Runs PostgreSQL for storing user data, quiz metadata, and analytics records.

Additionally, **LLM APIs (Google Gemini/OpenAI)** are accessed externally for AI-driven quiz generation. The backend communicates with these APIs through secure HTTPS connections. Optional **Docker containers** can encapsulate each node for scalable cloud deployment.

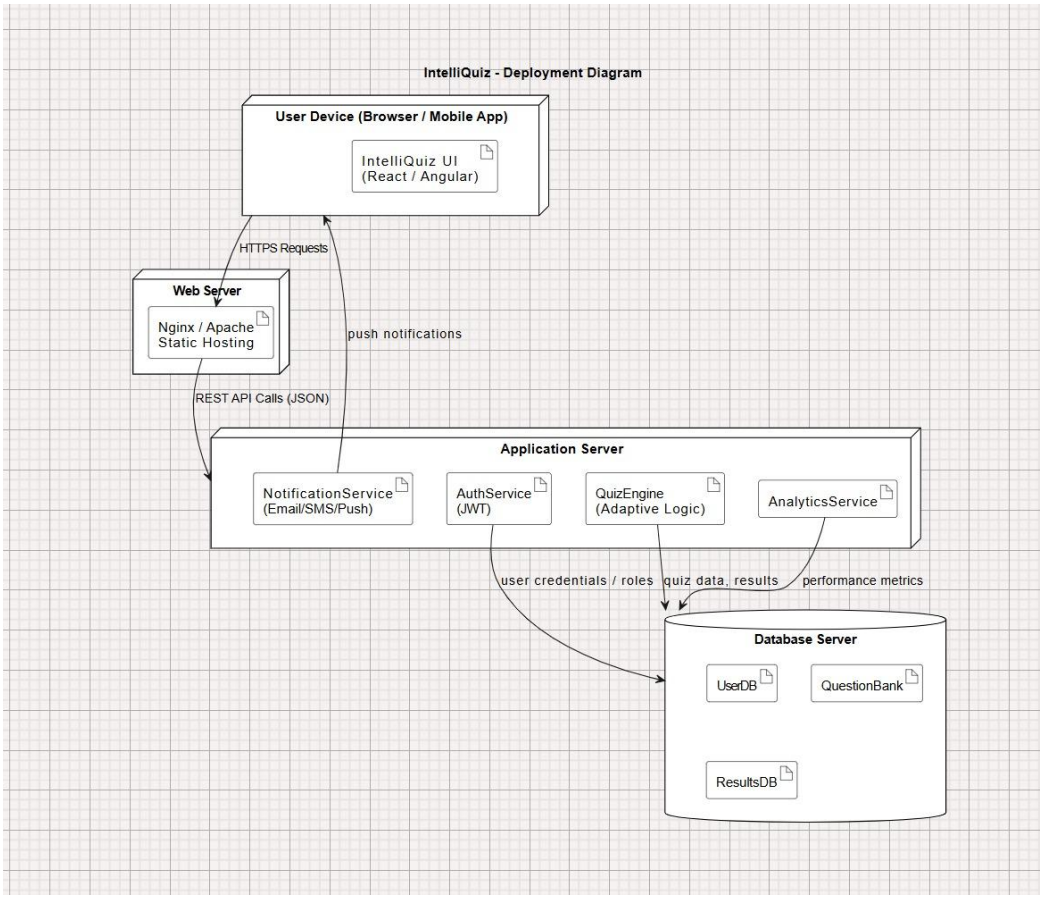

Fig-5: Deployment Diagram

4.6 System Architecture

The System Architecture Diagram provides a high-level overview of IntelliQuiz's integrated components and their interactions.

1. **Presentation Layer (Frontend):**

   Built with React.js and Tailwind CSS, this layer handles user interaction, quiz interfaces, and analytics visualization using Recharts.

2. **Application Layer (Backend):**

   Developed with Spring Boot, it consists of services for quiz management, user authentication (JWT-based), adaptive difficulty adjustment, and gamification logic.

3. **AI Integration Layer:**

   Communicates with Google Gemini or OpenAI APIs for question generation and validation. The Retrieval-Augmented Generation (RAG) model ensures contextual accuracy.

4. **Data Layer:**

   PostgreSQL stores all structured data — user details, questions, quiz results, and performance metrics.

5. **Analytics Layer:**

   Aggregates performance data and delivers interactive dashboards and reports.

Data flow is bidirectional between the frontend and backend through RESTful APIs, ensuring seamless synchronization between quiz interactions, user data, and analytics visualization.

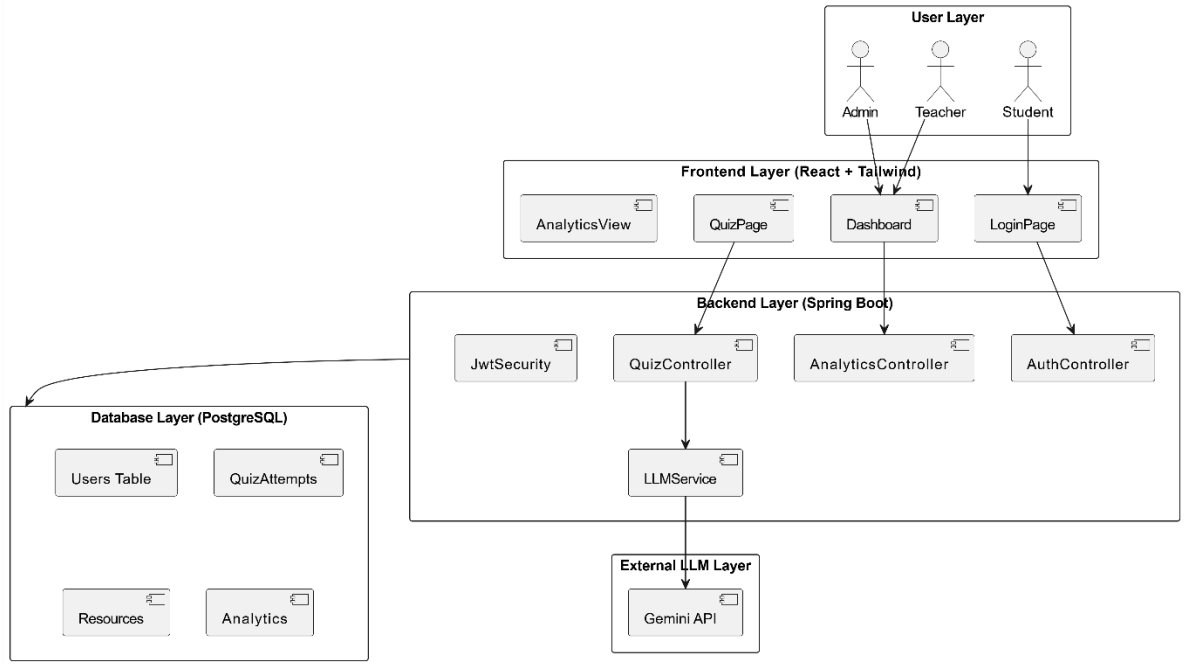

Fig-6: System Architecture

4.7 Data Flow Diagram

The **Data Flow Diagram (DFD)** traces the movement of information through IntelliQuiz's core modules.

Level 0 (Context Diagram):

- Inputs: Teacher uploads learning materials, Student attempts quiz.
- Processes: AI question generation, adaptive quiz delivery, scoring, and analytics.
- Outputs: Quiz results, leaderboards, and performance dashboards.

Level 1 (Detailed Flow):

1. **Upload Module** – Extracts textual content from teacher-uploaded resources.
2. **Question Generation Module** – Sends content to the LLM API for quiz creation.
3. **Adaptive Engine** – Selects appropriate questions dynamically during each student session.
4. **Evaluation Module** – Scores responses and updates gamification data.
5. **Analytics Module** – Compiles and visualizes performance data in real time.

Each module communicates through structured REST APIs, ensuring smooth and traceable data flow across the system.

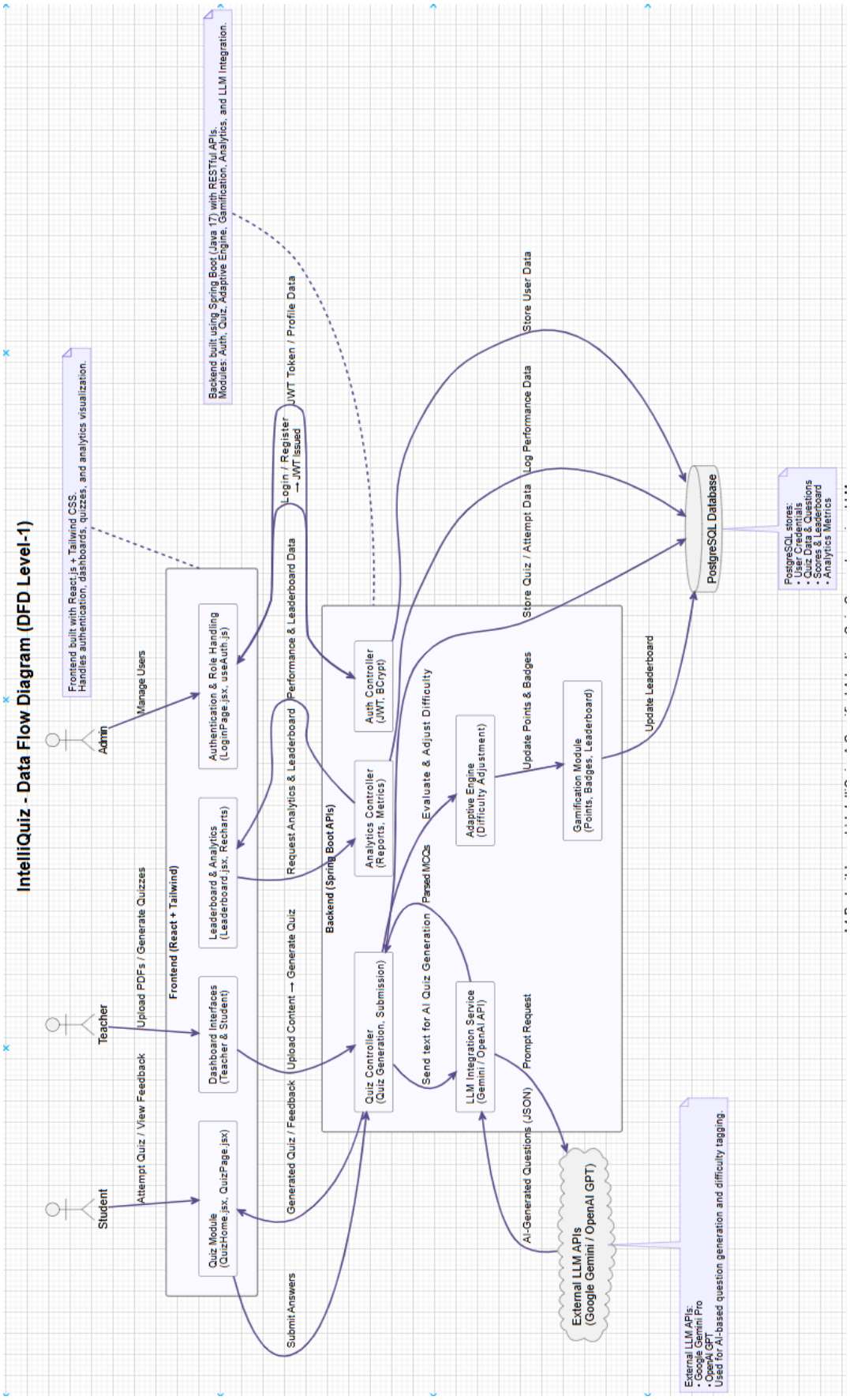

Fig-7: Data Flow Diagram

4.8 Activity Diagram

The Activity Diagram represents the process flow of how a student takes an adaptive quiz in **IntelliQuiz**. It shows the interaction between the user, frontend, and backend components during a quiz session.

1. The student opens the IntelliQuiz web application and logs in using JWT-based authentication.
2. If authentication succeeds, the system displays the available quizzes; otherwise, an error message is shown.
3. The student selects a quiz, and the system initializes an adaptive quiz session, fetching the first question from the AI-generated Question Bank.
4. The question is displayed, the student submits an answer, and the backend evaluates its correctness.
5. Based on performance, the system updates the score and adjusts the difficulty level dynamically.
6. This cycle repeats until all questions are completed.
7. After the quiz ends, the results are sent to the Analytics Service for evaluation.
8. The system updates the leaderboard and awards points and badges through the Gamification Module.
9. Finally, the student is notified of completion, and the results with feedback are displayed.

This diagram highlights the **adaptive quiz flow** in IntelliQuiz, demonstrating how the system integrates **AI-driven question selection, real-time evaluation, gamification, and analytics** to enhance learning engagement.

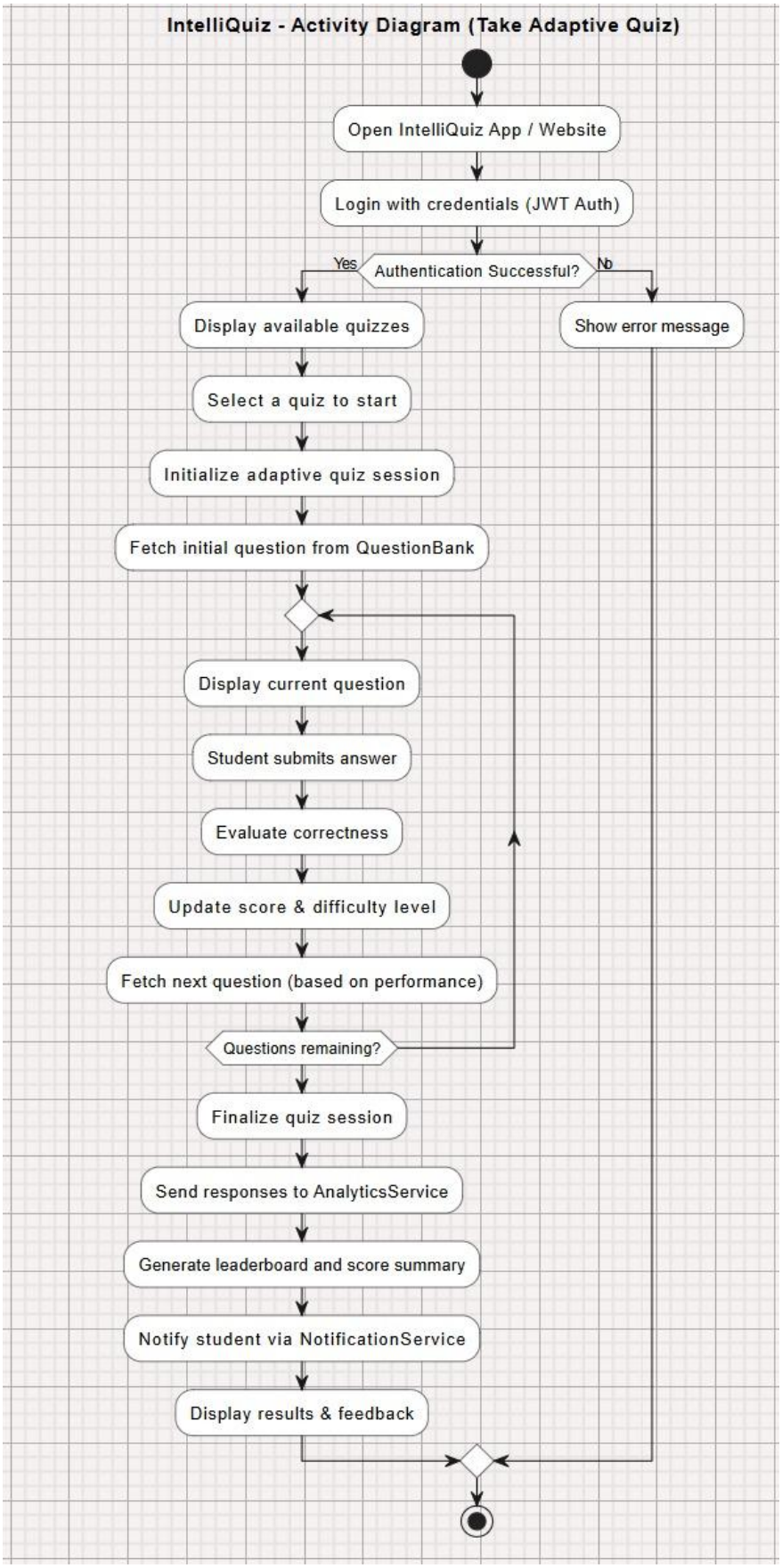

Fig-8: Activity Diagram

4.9 Component Diagram

- The Component Diagram shows the modular architecture of **IntelliQuiz**, divided into three main layers — **Frontend**, **Backend**, and **Data Layer**.
- The **Frontend Layer** includes the *IntelliQuiz UI*, which interacts with backend services through APIs such as **IAuthAPI**, **IQuizAPI**, **IAnalyticsAPI**, and **INotifyAPI**. The **Backend Layer** comprises key services like *AuthService*, *QuizEngine (Adaptive)*, *AnalyticsService*, and *NotificationService*, which handle authentication, quiz generation, adaptive logic, analytics, and user notifications. These services communicate with the **Data Layer** using the **IDataAPI** for accessing the *UserDB*, *QuestionBank*, and *ResultsDB*.
- This diagram highlights IntelliQuiz's **layered and modular design**, ensuring scalability, maintainability, and smooth communication between all components via structured APIs.

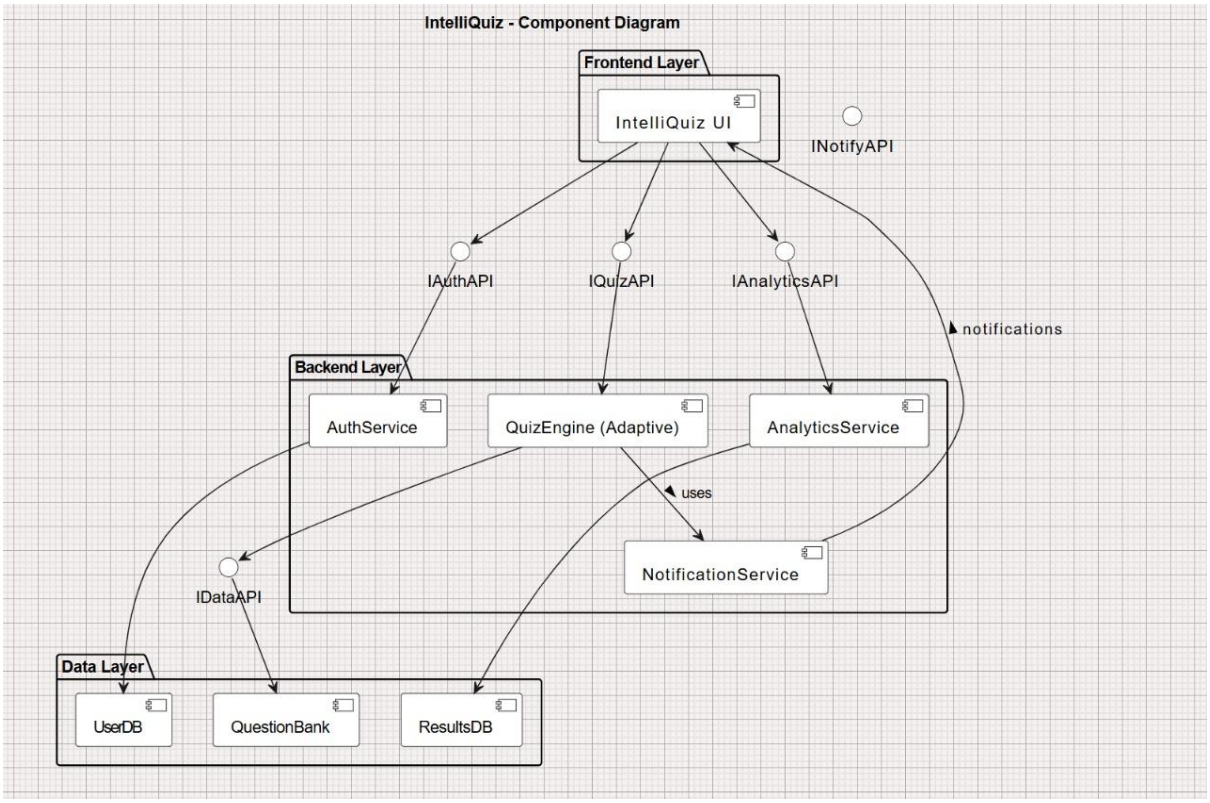

Fig-9: Component Diagram

5. IMPLEMENTATION

5.1 Overview

The **IntelliQuiz** system was implemented as a **modular, AI-driven, full-stack web platform** designed to automate quiz generation, enable adaptive assessments, and provide data-driven analytics.

The implementation uses a **React + Tailwind CSS** frontend, a **Spring Boot (Java 17)** backend, and a **PostgreSQL** database. The application also integrates **Google Gemini Pro** or **OpenAI GPT** APIs to generate multiple-choice questions dynamically from teacher-uploaded learning materials.

The design follows a **three-tier architecture**:

1. **Presentation Layer:** Frontend UI built with React JS for all user roles (Admin, Teacher, Student).
2. **Application Layer:** Backend microservices built with Spring Boot handling authentication, adaptive logic, and analytics.
3. **Data Layer:** PostgreSQL relational database managing user, quiz, and result data.

This modular structure ensures **scalability, reusability, and ease of maintenance**, allowing future integration with LMS platforms or mobile apps.

5.2 Repository Directory Structure

Defines paths to the modules and its associated data.

```
intelliquiz-backend/
│
├── .env
├── pom.xml
├── README.md
├── run.sh
│
├── data/
│   ├── uploads/                # PDF / video transcripts uploaded by teachers
│   ├── generated_quizzes/      # JSONs or logs of LLM-generated quizzes
│   ├── analytics_reports/      # User analytics exports / CSVs
│   ├── llm_prompts/            # Prompt templates and tuning examples
│   └── cache/                  # Cached embeddings / temporary data
│
├── logs/
│   └── app.log
│
├── modules/
│   ├── adaptive/
│   │   └── AdaptiveEngine.java
│   │
│   ├── llm/
│   │   ├── LLMService.java
│   │   ├── PromptTemplates.java
│   │   └── QuizPostProcessor.java
│   │
│   ├── analytics/
│   │   ├── AnalyticsService.java
│   │   └── TopicAnalyticsDTO.java
│   │
│   ├── quiz/
│   │   ├── QuizController.java
│   │   ├── QuizService.java
│   │   ├── GeneratedQuiz.java
│   │   └── QuizAttempt.java
│   │
```

```
|   ├── resources/
|   |   ├── ResourceController.java
|   |   ├── ResourceService.java
|   |   └── ResourceRepository.java
|   |
|   ├── user/
|   |   ├── AuthController.java
|   |   ├── UserDetailsImpl.java
|   |   ├── UserRepository.java
|   |   └── UserProfileService.java
|   |
|   └── classroom/
|       ├── ClassroomController.java
|       ├── ClassroomService.java
|       └── ClassroomRepository.java
|
├── config/
|   ├── SecurityConfig.java
|   ├── WebConfig.java
|   ├── DataInitializer.java
|   └── AppConstants.java
|
├── exceptions/
|   ├── GlobalExceptionHandler.java
|   ├── BadRequestException.java
|   └── TokenRefreshException.java
|
├── repository/
|   ├── QuizRepository.java
|   ├── UserRepository.java
|   ├── ResourceRepository.java
|   └── AttemptRepository.java
|
├── security/
|   ├── jwt/
|   |   ├── AuthTokenFilter.java
|   |   └── JwtUtils.java
|   ├── UserDetailsServiceImpl.java
|   └── CustomAuthEntryPoint.java
|
└── IntelliQuizApplication.java
```

Fig-10: Backend Repository Structure

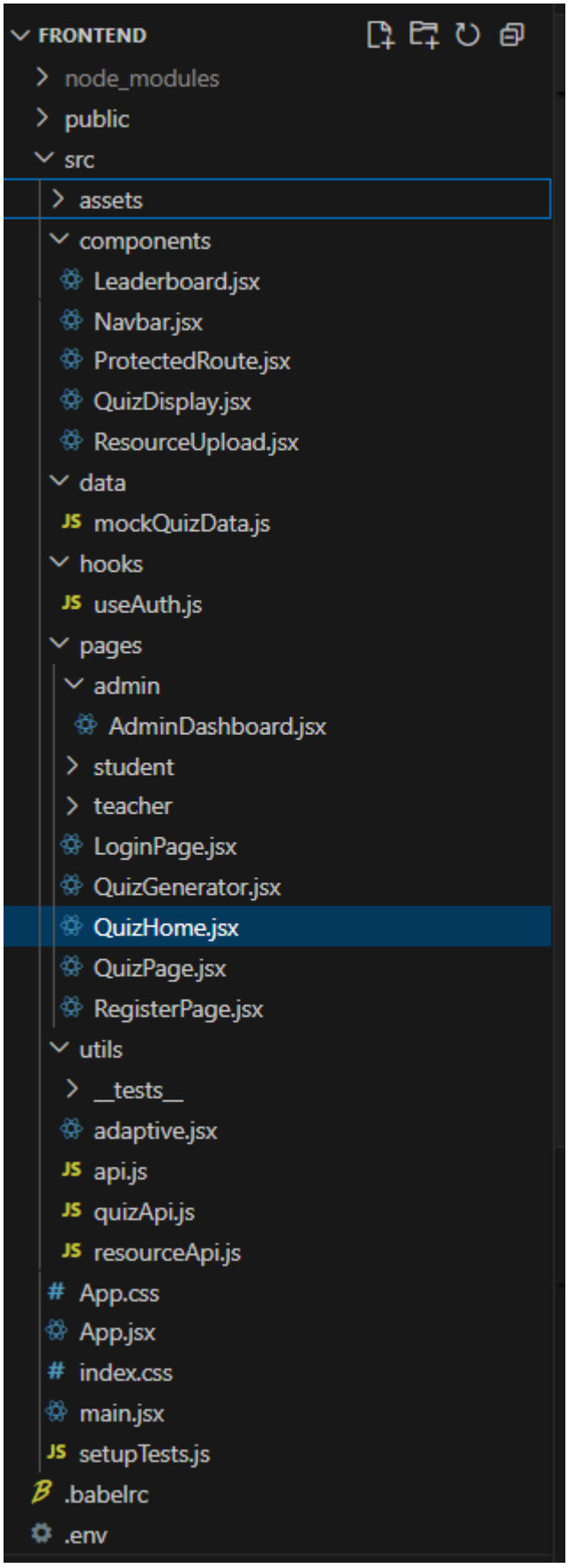

Fig-11: Frontend Repository Structure

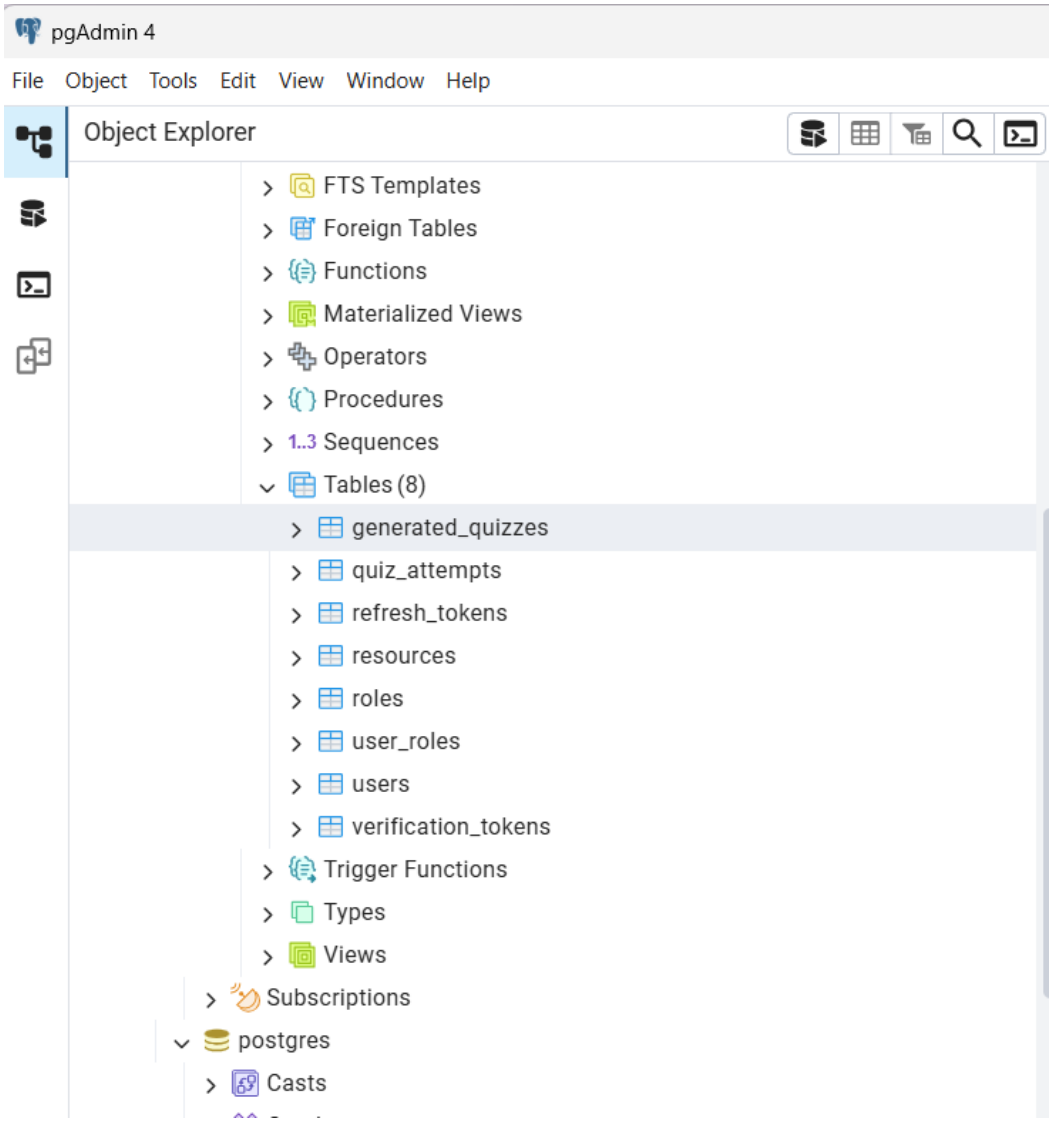

Fig. 12: Database repositories

5.3 Implementation by Module

The IntelliQuiz system is implemented as a modular, full-stack application composed of clearly defined backend micromodules and frontend interface components. Each module is responsible for specific functionality, enabling easy maintenance, scalability, and independent testing.

5.3.1 Backend Modules

The backend is built using **Spring Boot (Java 17)** and follows a REST-based layered architecture, where each layer encapsulates domain-specific logic and communicates via well-defined endpoints.

a) Authentication and Authorization Module

Objective:

Secure login, registration, and role-based access for students, teachers, and administrators.

Implementation Details:

- Controllers: AuthController.java handles endpoints /auth/login, /auth/register, and /auth/refreshtoken.
- JWT Security: Implemented via JwtUtils.java, AuthTokenFilter.java, and SecurityConfig.java.
- User authentication is validated through UserDetailsServiceImpl.java and token-based verification.
- Roles are defined in ERole.java (ROLE_ADMIN, ROLE_TEACHER, ROLE_STUDENT) and persisted via RoleRepository.java.
- Passwords are encrypted using **BCrypt**.

API Example:

POST /auth/login → Validates credentials and issues JWT.

POST /auth/register → Registers a user with default role (Student).

b) Resource Upload and Management Module

Objective:

Allow teachers and students to upload and manage learning materials (PDFs) which the LLM can later analyze to generate quizzes.

Implementation Details:

- Core classes: ResourceController.java, ResourceService.java, and ResourceRepository.java.
- File upload handled with MultipartFile, validated by type (PDF only).
- Uploaded resources are linked to topics and stored in uploads/ (configured in application.properties).
- Metadata (topic, uploader, file path) is persisted in PostgreSQL.
- PDF text extraction is managed by PdfService.java.

API Example:

POST /resources/upload → Uploads a PDF for a specific topic.
GET /resources/list → Lists all uploaded resources by user.
DELETE /resources/{id} → Deletes an existing resource.

c) LLM-Based Quiz Generation Module

Objective:

Automate the generation of context-aware, topic-aligned quizzes using AI models (Google Gemini / OpenAI GPT).

Implementation Details:

- Core service: LLMService.java interacts with Gemini API using configured keys.
- Receives cleaned content from PdfService.java, formats prompt templates, and sends them to the model endpoint.
- Returned data is parsed into structured question objects (GeneratedQuiz.java entity).
- Handles fallback when API latency or downtime occurs.

API Example:

POST /quiz/generate/ai → Generates MCQs for a given resource/topic.

GET /quiz/list → Retrieves all AI-generated quizzes.

GET /quiz/get/{id} → Fetches a quiz by its ID.

d) Adaptive Difficulty and Gamification Module

Objective:

Personalize the quiz difficulty based on user performance and increase motivation through gamified mechanics.

Implementation Details:

- Adaptive logic defined in AdaptiveEngine.java.
- Student accuracy, completion time, and score are analysed to adjust difficulty dynamically (Easy → Medium → Hard).
- Gamification points and badges are recorded in the database under each user.
- Integration points exist between QuizController.java and AnalyticsService.java for leaderboard updates.

e) Analytics and Reporting Module

Objective:

Provide teachers and students with real-time analytics on learning performance.

Implementation Details:

- Services: AnalyticsService.java with APIs exposed through AnalyticsController.java.
- Aggregates quiz data, performance trends, and leaderboard entries.
- Data formatted into DTOs: StudentAnalyticsResponse.java, ClassroomAnalyticsResponse.java, and LeaderboardEntryResponse.java.
- Teacher dashboards display individual and classroom trends.

5.3.2 Frontend Modules

The frontend is built using **React.js**, **Vite**, and **Tailwind CSS**, delivering a responsive single-page application with role-based dashboards. The structure follows a modular component hierarchy, improving readability and reusability.

a) Authentication and User Management Module

Components: LoginPage.jsx, RegisterPage.jsx, useAuth.js, ProtectedRoute.jsx.

Functionality:

- Provides user login, registration, and secure session handling via JWTs stored in localStorage.
- useAuth() manages token validation and auto-logout when tokens expire.
- ProtectedRoute ensures only authorized roles can access dashboards.

Flow:

Login → JWT retrieved → stored in localStorage → validated before each API call.

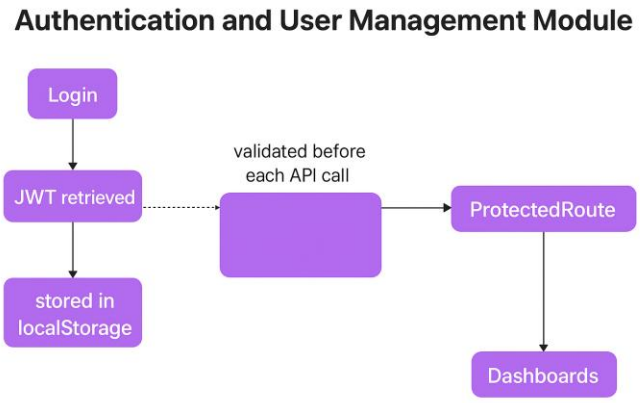

Fig. 13: Authentication and User Management Module

b) Student Dashboard and Resource Module

Components: StudentDashboard.jsx, ResourceUpload.jsx.

Functionality:

- Displays learning resources, uploads new PDFs, and generates quizzes from those resources.

- Integrates adaptive feedback (difficulty stored per topic).
- Shows performance data through **Recharts** graphs.

Workflow:

Upload Resource → AI Quiz Generated → Attempt Quiz → Analytics Updated.

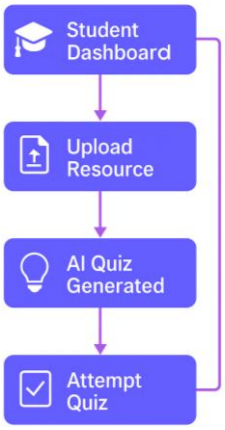

Fig. 14: Student and Resource Module

c) Quiz and Evaluation Module

Components: QuizHome.jsx, QuizGenerator.jsx, QuizPage.jsx, QuizDisplay.jsx.

Functionality:

- Quiz generation via AI (generateAIQuiz() in quizApi.js).
- Adaptive quiz rendering with navigation between questions.
- Submits quiz responses to backend via submitQuiz() API.

Features:

Real-time feedback, recommended difficulty adjustment, and score computation both server- and client-side.

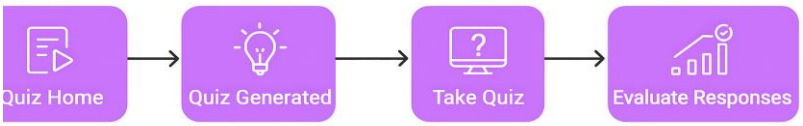

Fig. 16: Quiz and Evaluation Module

d) Leaderboard and Analytics Visualization Module

Components: Leaderboard.jsx, Navbar.jsx.

Functionality:

- Fetches leaderboard data (/analytics/leaderboard) and displays ranked students with points and badges.
- Fully responsive with color-coded rank indicators and animated progress bars.

Data Source:

Consumes JSON from AnalyticsController → displayed dynamically via React state hooks.

Fig. 15: Leaderboard and Analytics, Visualisation Module

5.4 Application Entry point

5.4.1 Backend Entry Point

The main entry to the IntelliQuiz backend is the **Spring Boot Application class**:

```
package com.intelliquiz.backend;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BackendApplication {

  public static void main(String[] args) {
    SpringApplication.run(BackendApplication.class, args);
  }

}
```

This initializes all configurations:

- Loads database and LLM configurations from application.properties.
- Initializes beans for services (LLMService, AnalyticsService, etc.).

- Starts embedded **Tomcat server** on port 8080.

Upon startup, the *DataInitializer.java* seeds default roles (Admin, Teacher, Student) into the PostgreSQL database if they do not exist.

5.4.2 Frontend Entry Point

The React application starts from the **main.jsx** file:

This initializes:

- **React Router** for client-side navigation.
- **Protected Routes** for role-based dashboards (Student, Teacher, Admin).
- Global toast notifications for success/error handling.

The first page loaded is /login, redirecting users post-authentication to their respective dashboards.

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter, Routes, Route, Navigate, useLocation } from
"react-router-dom";
import "./index.css";
import "react-toastify/dist/ReactToastify.css";
import Navbar from "./components/Navbar";
import ProtectedRoute from "./components/ProtectedRoute";
import Leaderboard from "./components/Leaderboard";
import { ToastContainer } from "react-toastify";
import LoginPage from "./pages/LoginPage";
import RegisterPage from "./pages/RegisterPage";
import StudentDashboard from "./pages/student/StudentDashboard";
import TeacherDashboard from "./pages/teacher/TeacherDashboard";
import AdminDashboard from "./pages/admin/AdminDashboard";
import QuizHome from "./pages/QuizHome";
import QuizPage from "./pages/QuizPage";
import QuizGenerator from "./pages/QuizGenerator";
function Layout() {
  const location = useLocation();
  const hideNavbarRoutes = ["/login", "/register"];
  return (
    <>
      {!hideNavbarRoutes.includes(location.pathname) && <Navbar />}
      <Routes>
        <Route path="/" element={<Navigate to="/login" replace />} />
        <Route path="/login" element={<LoginPage />} />
        <Route path="/register" element={<RegisterPage />} />
        <Route path="/student/dashboard" element={
            <ProtectedRoute
requiredRoles={["ROLE_STUDENT"]}><StudentDashboard /></ProtectedRoute>
```

```
          }/>
        <Route path="/student/quizzes" element={
            <ProtectedRoute requiredRoles={["ROLE_STUDENT"]}><QuizHome
/></ProtectedRoute>
          }/>
        <Route path="/quiz" element={
          <ProtectedRoute requiredRoles={["ROLE_STUDENT"]}><QuizPage
/></ProtectedRoute>
          }/>
        <Route path="/quiz/start" element={
            <ProtectedRoute requiredRoles={["ROLE_STUDENT"]}> <QuizPage
/> </ProtectedRoute>
          }/>
        <Route path="/quiz/generate" element={
            <ProtectedRoute
requiredRoles={["ROLE_STUDENT"]}><QuizGenerator /></ProtectedRoute>
          }/>
        <Route path="/leaderboard" element={
            <ProtectedRoute><Leaderboard /></ProtectedRoute>
          }/>
        <Route path="/teacher/dashboard" element={
            <ProtectedRoute requiredRoles={["ROLE_TEACHER"]}>
<TeacherDashboard /></ProtectedRoute>
          }
        />
        <Route path="/admin/dashboard" element={
            <ProtectedRoute requiredRoles={["ROLE_ADMIN"]}>
<AdminDashboard /> </ProtectedRoute>
          }/>
        <Route path="*" element={<Navigate to="/student/dashboard"
replace />} />
      </Routes>
    </>
  );
}
// ☑ No export — just render the app
ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <Layout />
      <ToastContainer position="bottom-right" autoClose={3000} />
    </BrowserRouter>
  </React.StrictMode>
);
```

This initializes:

- **React Router** for client-side navigation.
- **Protected Routes** for role-based dashboards (Student, Teacher, Admin).
- Global toast notifications for success/error handling.

The first page loaded is /login, redirecting users post-authentication to their respective dashboards.

5.5 Dependencies & Environment

5.5.1 Backend Dependencies

The IntelliQuiz backend utilizes several core libraries and frameworks defined in pom.xml:

Dependency	Purpose
Spring Boot Starter Web	Core framework for REST APIs
Spring Boot Starter Security	Authentication and JWT support
Spring Boot Starter Data JPA	Database ORM with Hibernate
PostgreSQL Driver	Database connection
Lombok	Code simplification (getters/setters)
Gson / Jackson	JSON parsing
Apache POI & PDFBox	PDF parsing and content extraction
OkHttp / RestTemplate	External API (LLM) integration
JUnit 5	Testing framework

- **Database:** PostgreSQL v16
- **Server Port:** 8080
- **API Keys:** Configured in application.properties under gemini.api.key and openai.api.key

5.5.2 Frontend Dependencies

Configured via `package.json` (Vite-based build):

Package	Purpose
React, React DOM	Frontend framework
React Router DOM	Routing and navigation
Axios	REST API communication
Framer Motion	UI animations
Recharts	Data visualization
Tailwind CSS	Styling framework
React Toastify	Notifications
Lucide React	Icon pack
Vite	Fast build tool

5.5.3 Environment Setup

Backend Environment:

- JDK 17
- Maven 3.8+
- PostgreSQL Server running on port 5432
- application.properties contains connection credentials

Start Command:

mvn spring-boot:run

Frontend Environment:

- Node.js 18+
- Install dependencies:

npm install

- Run development server:

npm run dev

- Accessible at: **http://localhost:5173**

6. SOFTWARE TESTING

Software testing for IntelliQuiz focuses on ensuring correctness, reliability, security, performance, and usability across the full stack: React frontend, Spring Boot backend, PostgreSQL persistence, LLM integrations, and the analytics/gamification modules. Tests are organized into Unit, Integration, Functional, Security, Performance, and System acceptance tests. Wherever external LLM APIs are involved, we rely on mocking/caching strategies to enable deterministic automated testing.

6.1 Test Objectives

- Verify functional correctness of core modules (UserAuth, QuizGen, AdaptiveEngine,
- Scoring, Gamification, Analytics).
- Ensure API contracts and frontend-backend interactions are stable.
- Validate resilience and graceful degradation when LLM APIs fail or are slow.
- Ensure acceptable performance under expected academic loads.
- Validate security (authentication, authorization, input sanitization, prompt-injection mitigations).
- Ensure accessibility and usability of the UI.

6.2 Testing Strategy & Tools

- **Unit / Integration (Backend):** JUnit 5, Mockito, Spring Boot Test, Testcontainers (for ephemeral PostgreSQL).
- **Unit / Integration (Frontend):** Jest, React Testing Library, MSW (Mock Service Worker) for API mocks.
- **E2E / Functional:** Cypress (UI end-to-end flows).
- **API / Contract Testing:** Postman Collections + Newman or Pact (consumer-driven contract tests).
- **Performance / Load:** Gatling or Apache JMeter (simulate concurrent users).
- **Security Scans:** OWASP ZAP (dynamic scan), Snyk/Dependabot (dependency vulnerability scanning).
- **Accessibility:** axe-core (integrated into Cypress).
- **CI/CD Integration:** GitHub Actions (run unit tests, static analysis, contract tests, and build).
- **Mocking LLMs:** Local mock server (WireMock / MSW) and cached responses for deterministic tests.
- **Observability:** Prometheus + Grafana (test run metrics), ELK or Loki for logs.

6.3 Unit Testing

Goals

Test individual units in isolation (services, controllers, repository layer).

Achieve high logic coverage for critical modules: AdaptiveEngine, QuizGenerator, ScoringCalculator, AuthService.

Backend examples & focus

AuthControllerTest (JUnit + Mockito): test registration, login, JWT issuance, role mapping, password hashing (BCrypt)

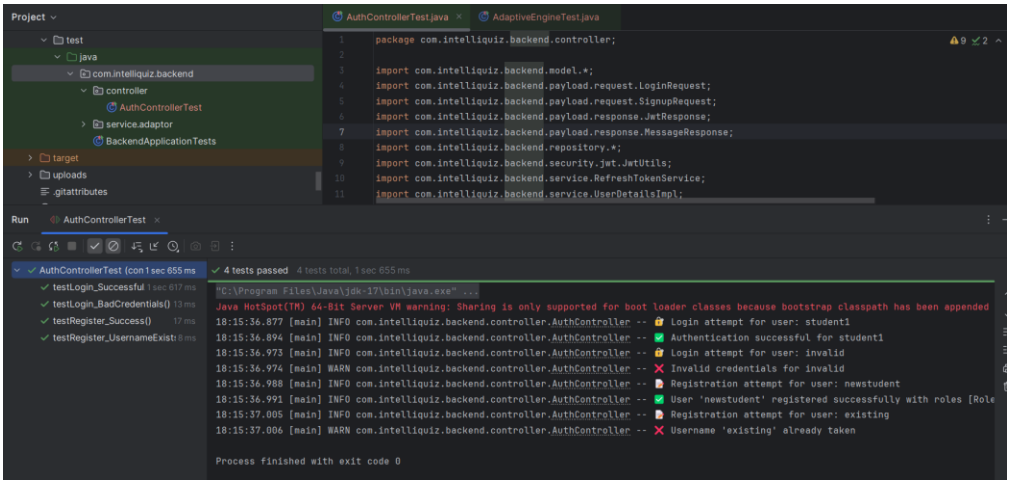

Fig.17: AuthControllerTest

LLMServiceTests: validate prompt construction, context-selection logic (top-k retrieval), candidate question filtering logic (without calling external LLM). Use mocks for RAG components and LLM client.

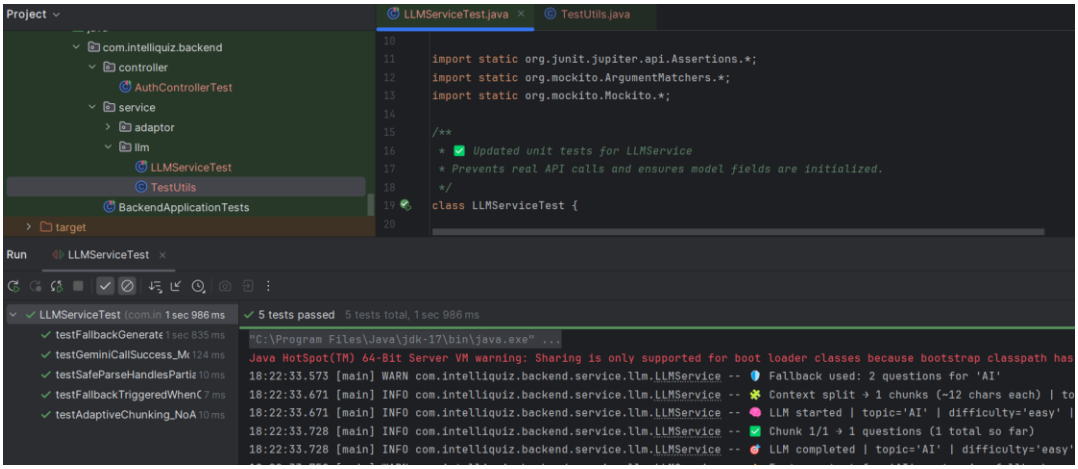

Fig.18: LLMServiceTests

AdaptiveEngineTests: verify difficulty updating logic, level-up/level-down thresholds, and question selection based on student history.

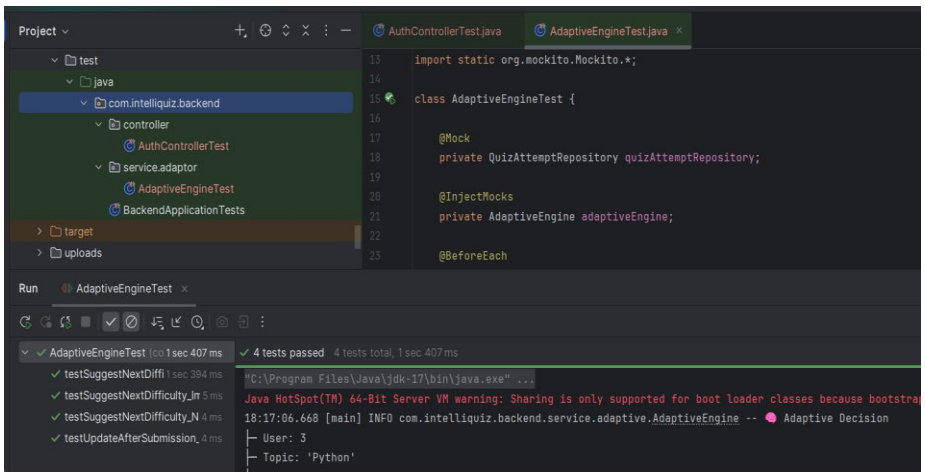

Fig.19: AdaptiveEngineTests

ScoringServiceTests: validate final score formula (weights for accuracy, time, partial credit), edge cases (no answer, timeouts).

Repository Tests: use Testcontainers PostgreSQL to test JPA entity persistence and repository queries.

Frontend tests

Component Unit Tests: React components (QuizAttemptCard, Leaderboard, AnalyticsChart) with Jest + React Testing Library.

Service Layer Tests: API client layer with MSW to simulate API responses (success, 4xx, 5xx).

State Management: verify Redux/Context updates for scoring and user session.

Acceptance criteria

Unit tests pass on every PR.

Target ≥80% coverage for business-critical backend classes; ≥60% for frontend components.

6.4 Integration Testing

Goals

Verify interactions between multiple components (e.g., frontend → backend → DB, backend → LLM client → DB).

Validate persistence, messaging/state changes, and data flows.

Approach

Spring Boot integration tests with @SpringBootTest and Testcontainers (Postgres).

Contract tests using Pact or Postman: ensure frontend assumptions about backend APIs remain valid.

LLM integration layer: run with mocked LLM endpoints (WireMock) for CI; perform scheduled staging runs against real LLMs with limited quota and Canary keys.

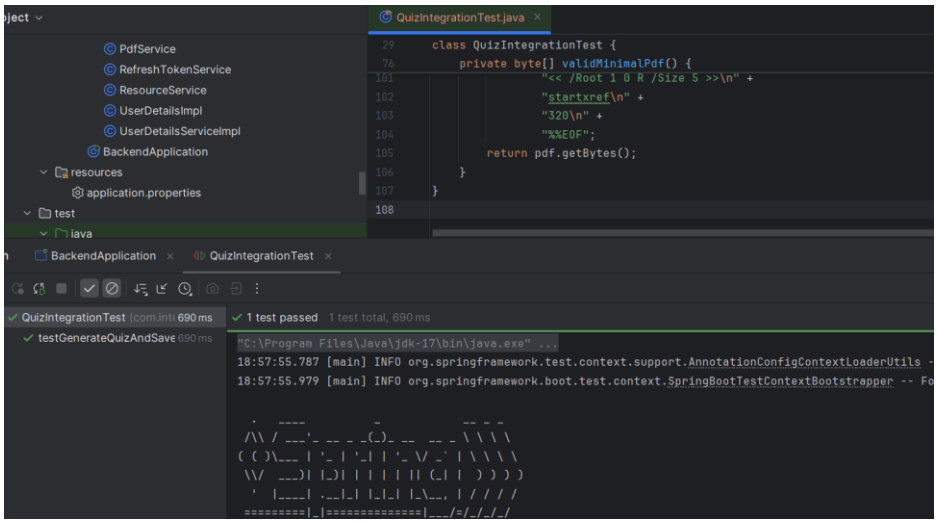

Fig. 20: test validates the complete pipeline from PDF upload → parsing → mocked AI quiz generation → database persistence, returning an HTTP 200 OK response.

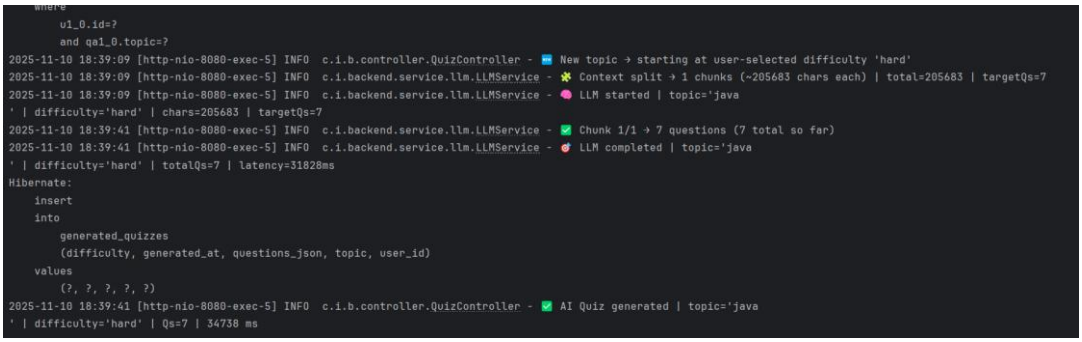

Fig. 21: Backend console log showing LLMService quiz generation and persistence workflow

Sample scenarios

Generate Quiz Flow: Teacher uploads PDF → Backend invokes ingestion → RAG returns top-k chunks → LLM mock returns questions → Questions saved to DB → Frontend fetches quiz list. Assert persisted question count, tags, and difficulty labels.

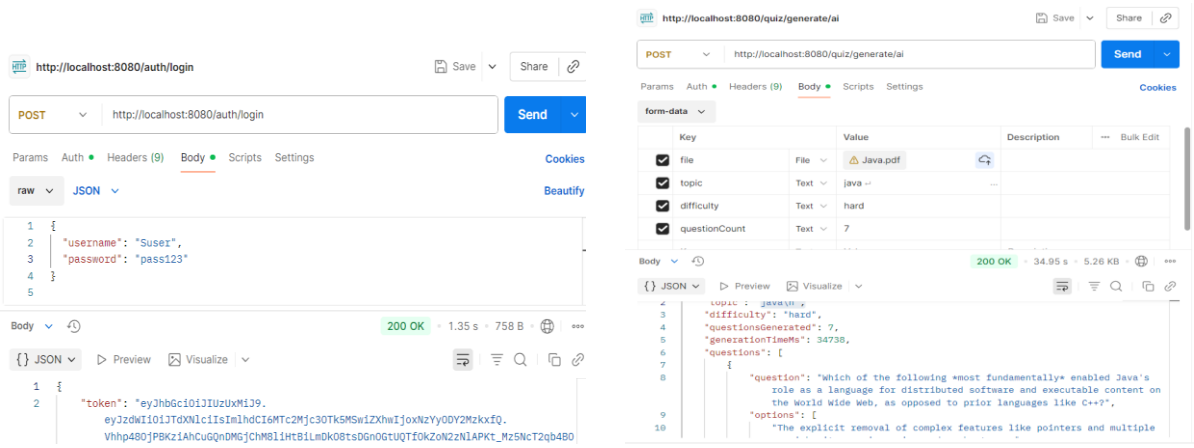

Fig. x: Integration test in Postman verifying successful AI-based quiz generation endpoint.

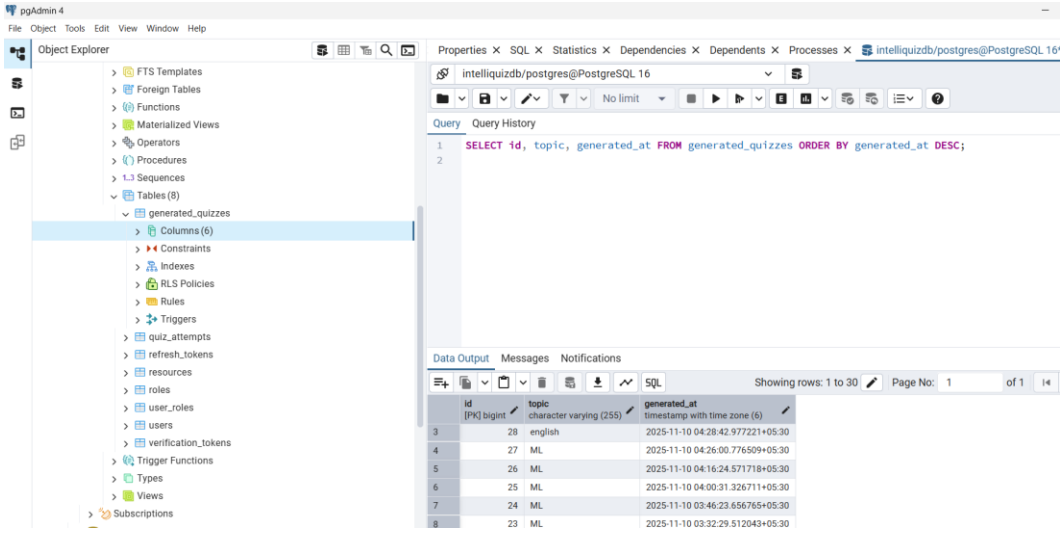

Fig. 22: PostgreSQL database entry showing newly persisted AI-generated quiz record

Attempt & Score Flow: Student fetches quiz → attempts questions → backend scoring computes score and updates gamification points → leaderboard updated. Assert expected points, badges, and analytics entries.

6.5 Functional Testing

Functional testing of IntelliQuiz was carried out from the **end-user perspective** to validate the correct working of all major system flows. The goal was to ensure that the integrated system functions according to the defined requirements across all user roles — **Admin, Teacher, and Student**.

Goals

The functional testing phase aimed to:

- Validate end-to-end features from a user's perspective including registration/login, quiz creation and assignment, quiz attempt, analytics visualization, and leaderboard behavior.
- Ensure that the UI interacts seamlessly with backend APIs and that the expected results are displayed without errors or data loss.
- Verify adaptive difficulty and gamification logic within real quiz sessions.

Testing Approach

- Manual testing was performed across all three user roles using the deployed IntelliQuiz web application and API verification via Postman.
- Each role was tested through realistic workflows using pre-created sample accounts in the PostgreSQL database.

Test Case 1 – User Login:

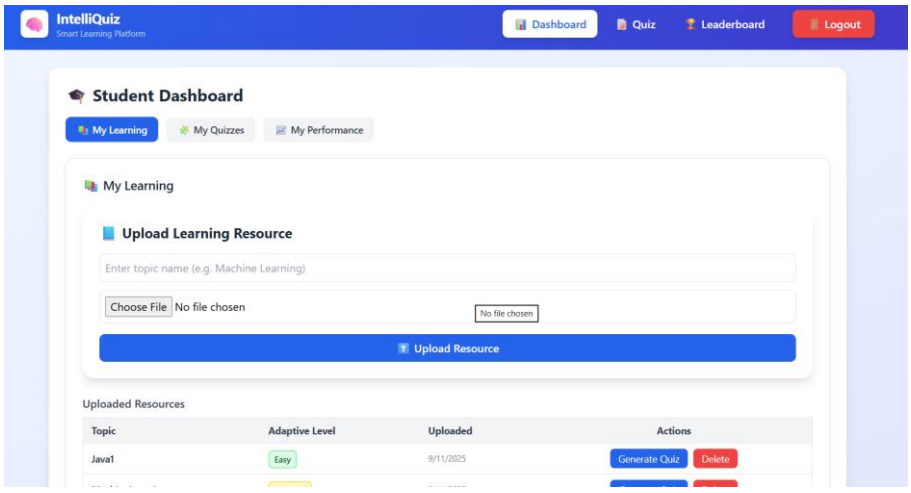

Fig.23: *User Login [student login → Dashboard redirects]*

Test Case 2 – Upload Resource & Generate Quiz:

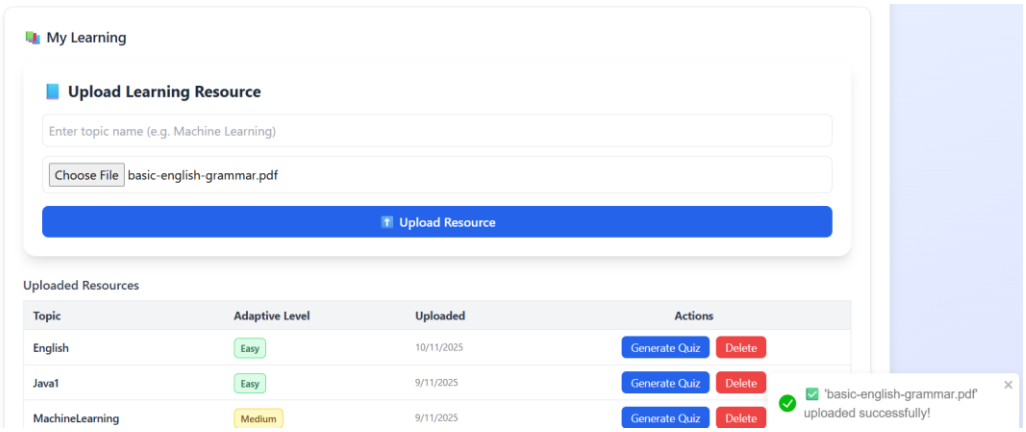

Fig.24: *Resource upload → success toast.*

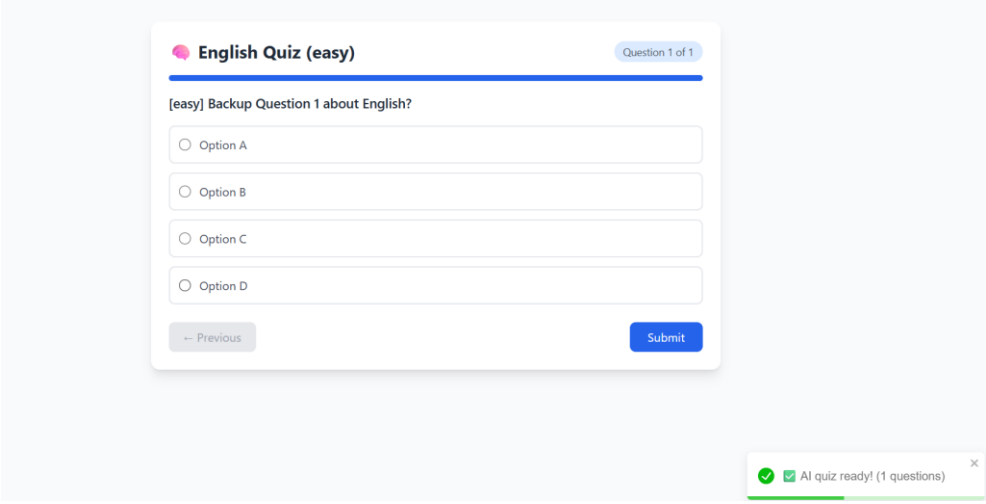

Fig.25: Generated quiz preview with questions.

Acceptance Criteria

All critical end-to-end flows passed during functional testing, ensuring that:

- System functions remained stable during real interactions.
- All UI components displayed accurate, consistent data from the backend.
- Adaptive difficulty logic and gamification elements behaved as intended.
- Analytics graphs and reports were rendered dynamically and error-free.

6.6 Security Testing

Goals

Ensure authentication/authorization, protect sensitive data, prevent injection and prompt-injection attacks.

Checks

Authentication & RBAC: attempt unauthorized endpoint access for each role; expected 401/403 responses.

Input validation & sanitization: test text uploads containing scripts, SQL meta-chars, and escape sequences; expect sanitized or rejected inputs.

Prompt injection tests: craft student-submitted content attempting to override system prompts; LLM prompt templates must be immune (system prompts appended, user content sanitized/escaped).

Dependency scan: automated vulnerability checks via Snyk or GitHub Dependabot.

Dynamic scanning: run OWASP ZAP against staging to find XSS, CSRF, open redirects.

Secrets management audit: verify no API keys in source; keys loaded from secure env (secrets manager in production).

Acceptance criteria

No critical or high vulnerabilities unresolved on releases.

All role-based access tests must pass.

6.7 Performance & Load Testing

Goals

Validate system behavior under expected and peak loads and identify bottlenecks.

Scenarios & metrics

Concurrent Users: simulate 5,000 concurrent students (read-heavy) and 1,000 concurrent quiz attempts (write-heavy).

LLM Latency simulation: simulate LLM response times (50ms–4s) and verify graceful degradation (cached Qs served when LLM slow).

Throughput: target 200 TPS for read APIs; 50 TPS for write APIs under sustained load.

SLAs: 95th percentile response time < 2s for dashboard queries; quiz generation < 15s (using cached contexts and parallel calls).

Tools & approach

Use **Gatling/JMeter** for load injection.

Measure CPU, memory, DB connection pool saturation, and queue lengths.

Run tests in staging using representative dataset sizes (100k questions, 10k users).

Acceptance criteria

System gracefully handles target load with <5% error rate.

LLM fallbacks reduce error spikes when external API latency increases.

6 RESULTS

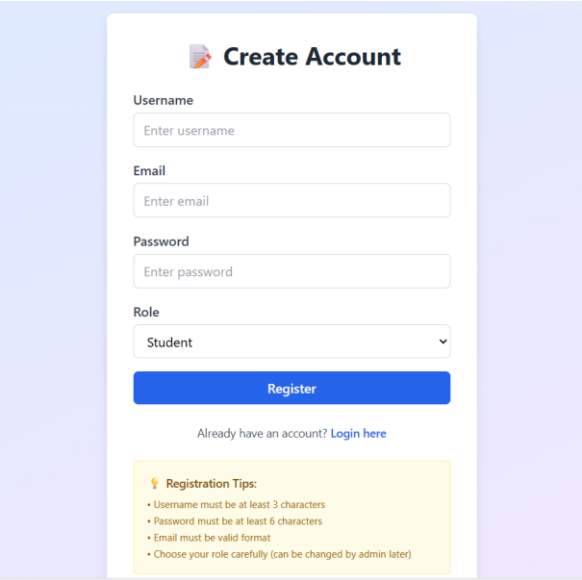

Fig-26: Registration Page

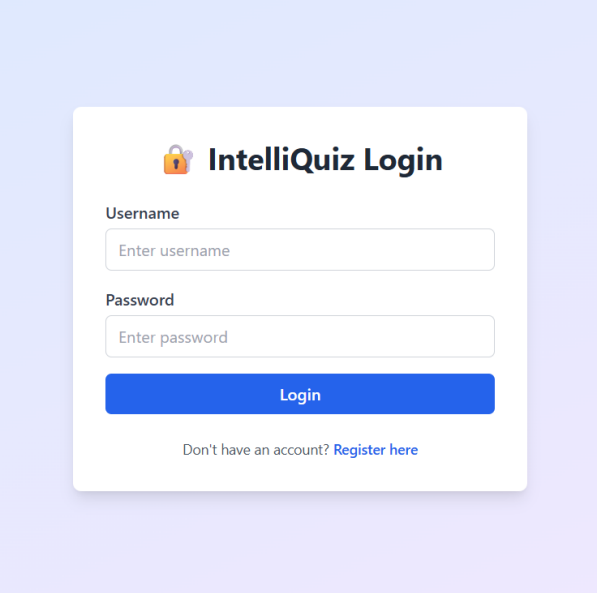

Fig-27: Login Page

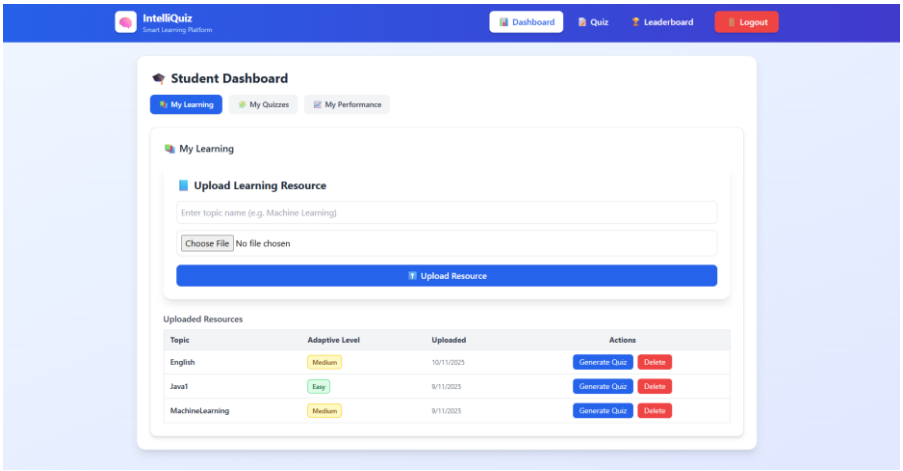

Fig-28: Student Dashboard

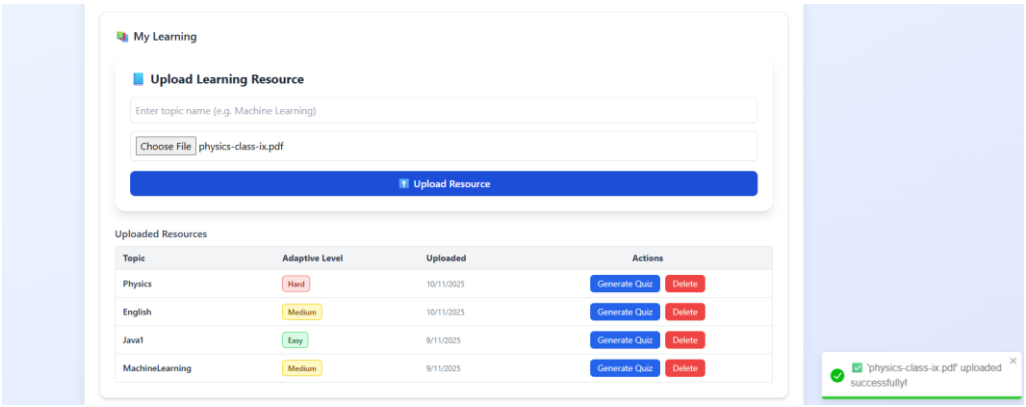

Fig-29: Upload Resource

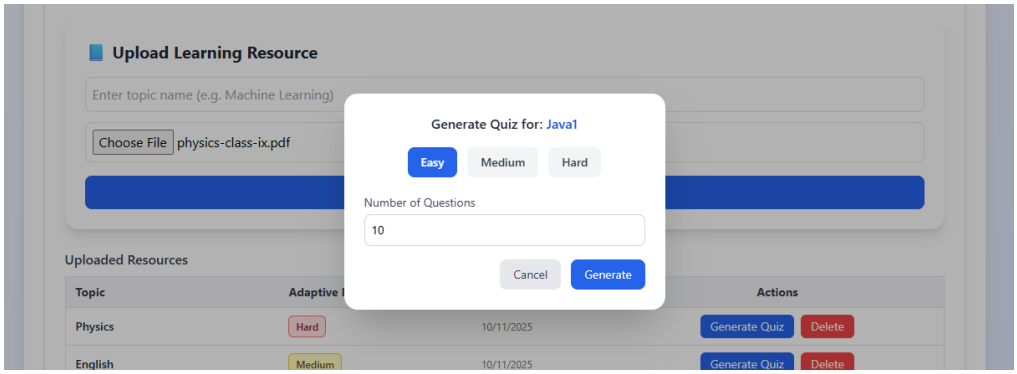

Fig-30: On clicking generate Quiz on uploaded Resources

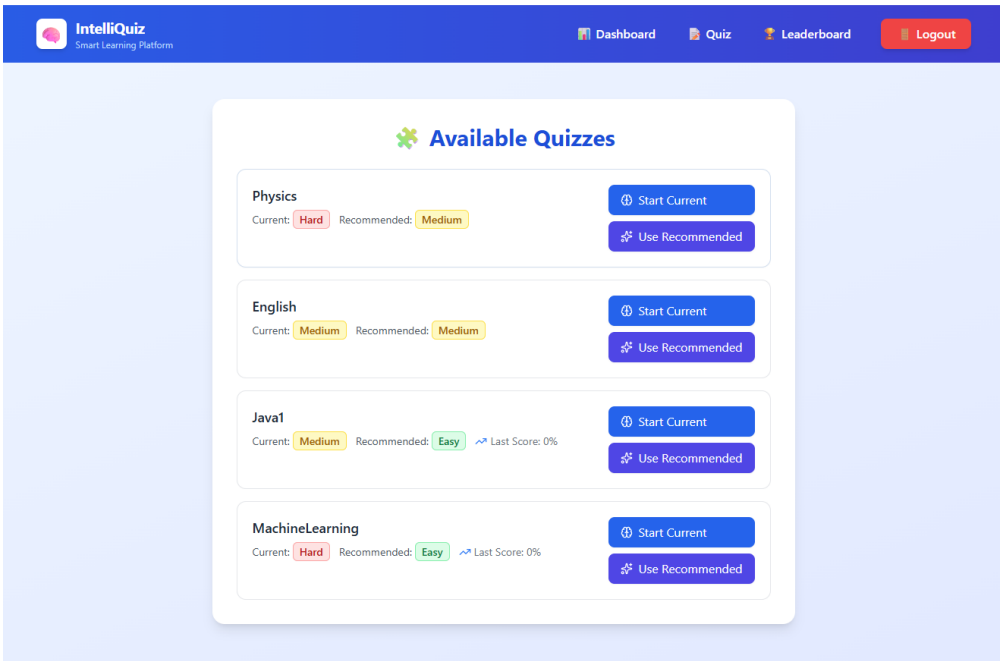

Fig-31: Available quizzes on quiz page

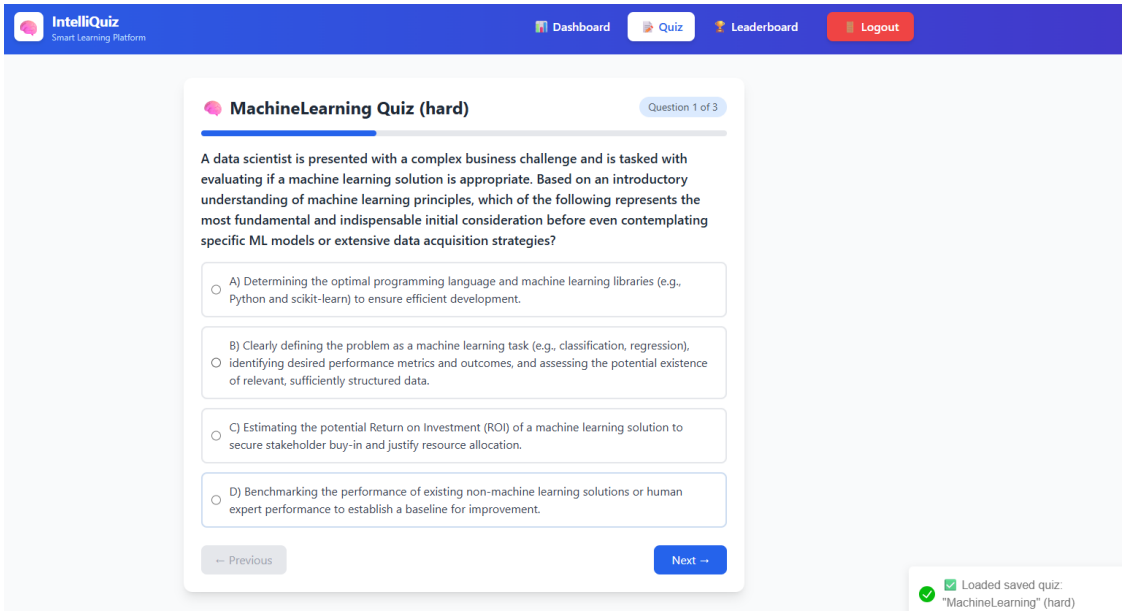

Fig-32: Answers generated and loaded in quiz page

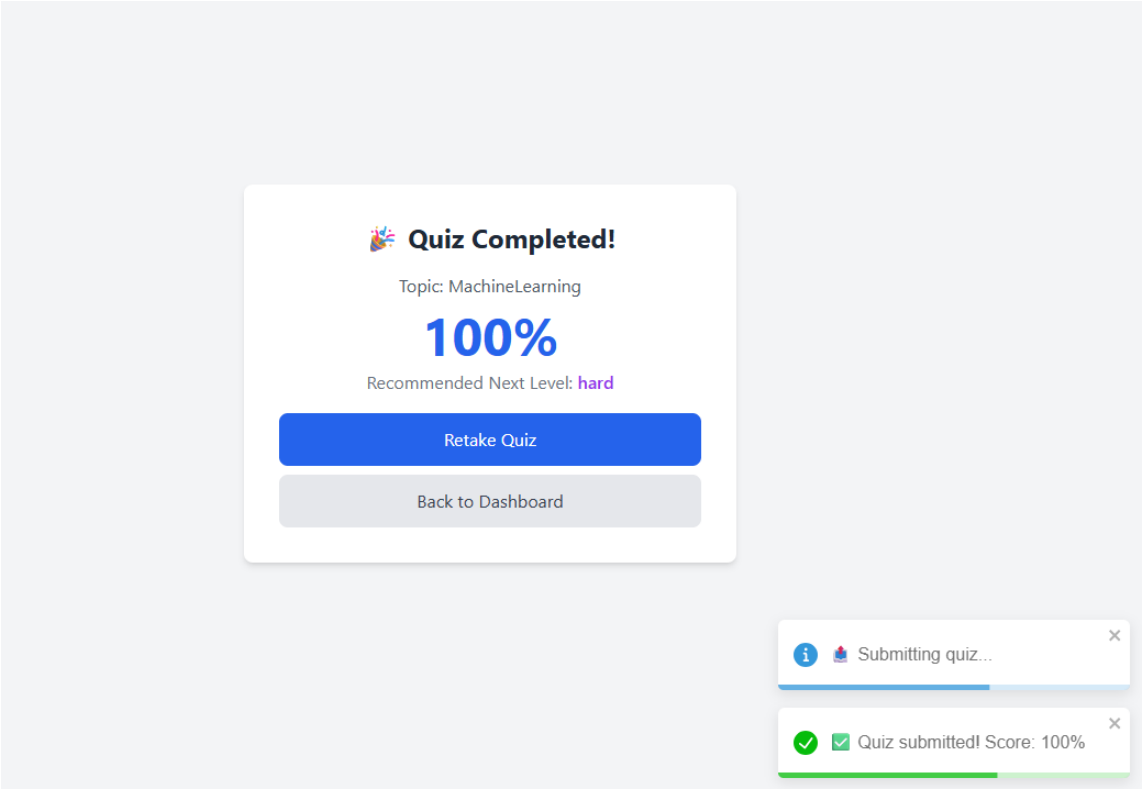

Fig-33: Quiz Submission and evaluation

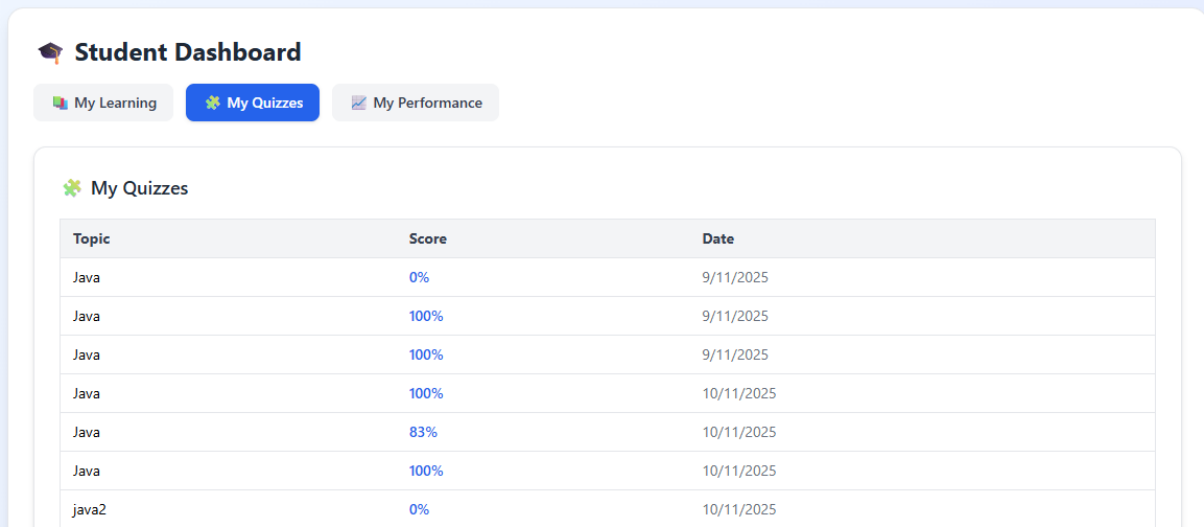

Fig-34: Student Attempted Quizzes [My Quizzes]

Fig-35: Student Performance history [My Performance]

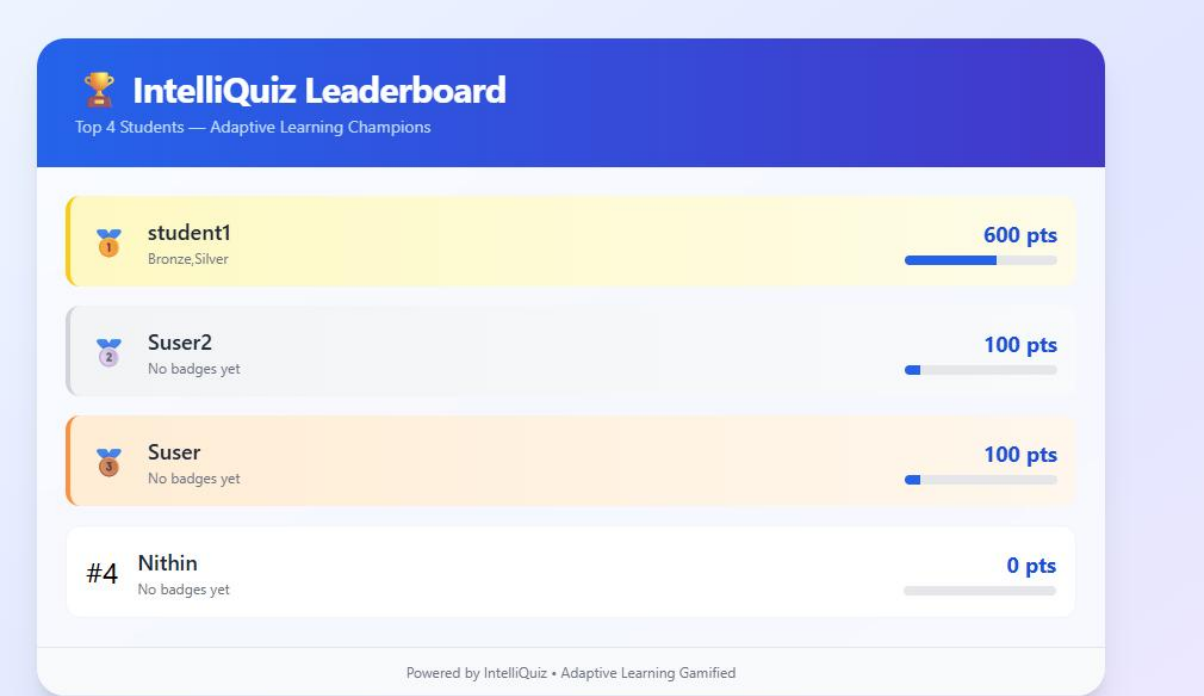

Fig-36: Leaderboard of students

7 CONCLUSION AND FUTURE SCOPE

Conclusion

The **IntelliQuiz** project successfully demonstrates how Artificial Intelligence, particularly **Large Language Models (LLMs)**, can revolutionize the way assessments and learning are conducted in modern educational environments. Traditional quiz systems often lack personalization, engagement, and scalability, but IntelliQuiz bridges these gaps through a unified platform that combines **adaptive learning**, **gamification**, and **real-time analytics**.

By integrating **Google Gemini Pro** and **OpenAI APIs** for automatic quiz generation, the system can create multi-level, context-aware MCQs from diverse content sources such as PDFs, video transcripts, and text. The **Adaptive Engine** ensures that quiz difficulty evolves dynamically based on each learner's performance, while the **Gamification Module**—featuring points, badges, and leaderboards—enhances motivation and sustained engagement. Teachers gain powerful insights through the **Analytics Dashboard**, which visualizes topic-wise performance, student progress, and class trends, enabling data-driven academic interventions.

Technically, IntelliQuiz is built on a robust and scalable architecture using **Spring Boot**, **React.js**, and **PostgreSQL**, ensuring seamless interaction between backend services, frontend dashboards, and AI APIs. The system also incorporates **role-based authentication**, **cloud deployment readiness**, and **security best practices**, making it adaptable for institutions of varying sizes.

Through IntelliQuiz, we have achieved a smart, interactive, and adaptive assessment environment that minimizes manual effort, improves feedback speed, and personalizes learning pathways. The successful implementation of this project marks a step forward in intelligent educational technology and lays a strong foundation for future innovations in AI-driven e-learning.

Future Scope

While IntelliQuiz has met its primary objectives, there are several directions for future enhancement to make the system more comprehensive, intelligent, and inclusive:

1. **Enhanced Adaptive Algorithms:**

   Implement deeper learner modeling using Reinforcement Learning (RL) or Bayesian Knowledge Tracing (BKT) to improve accuracy in predicting learner proficiency and tailoring question selection.

2. **Multi-Modal Content Generation:**

   Extend question generation beyond text to include **images, diagrams, and audio-visual comprehension questions**, utilizing multi-modal LLMs to support broader subject domains.

3. **Offline and Edge AI Support:**

   Introduce on-device inference capabilities or lightweight quantized models to allow IntelliQuiz to function efficiently in low-connectivity environments such as rural institutions.

4. **Expanded Gamification Features:**

   Incorporate **peer challenges, collaborative quizzes, and classroom competitions** to promote social learning and teamwork among students.

5. **Integration with Learning Management Systems (LMS):**

   Connect IntelliQuiz with popular LMS platforms like Moodle, Google Classroom, or Canvas to enable seamless data exchange, grading synchronization, and wider institutional adoption.

6. **AI-Powered Feedback and Explanation Engine:**

   Provide detailed, concept-level explanations for wrong answers using LLM-based reasoning, enabling learners to understand not just *what* they got wrong, but *why*.

7. **Enhanced Security and Academic Integrity:**

   Integrate **plagiarism and AI misuse detection**, secure proctoring features, and behavior-based anomaly detection to maintain fairness and trust in AI-driven assessments.

8. **Longitudinal Learning Analytics:**

   Develop predictive analytics dashboards that track learning trends over time and suggest targeted interventions for struggling students or low-performing topics.

8 REFERENCES

1) M. Chaudhary, A. Mahajan, S. Sharma, and S. Kumar, "Intelligent Integrated Knowledge Discovery Platform: Advancements in Question Generation and Adaptive Learning," *Journal of Educational Systems Research*, 2025. [Online]. Available: https://journal.esrgroups.org/jes/article/download/8690/5810/15789
2) G. Singaravel, M. Gobinath, S. R. Menaka, and S. Suganya, "Smart MCQ Generator for Personalized Learning Paths Using RAG and Generative AI," *International Journal of Creative Research Thoughts*, 2025. [Online]. Available: https://ijcrt.org/papers/IJCRT2506369.pdf
3) D. Sreekanth, "UC-100 Agentic AI Quiz Generation: Personalized Tutoring through Intelligent Retrieval and Adaptive Learning," *Kennesaw State University*, 2025. [Online]. Available: https://www.scribd.com/document/901461552
4) S. Maity and A. Deroy, "The Future of Learning in the Age of Generative AI: Automated Question Generation and Assessment with LLMs," *arXiv preprint arXiv:2410.09576*, 2024. [Online]. Available: https://arxiv.org/abs/2410.09576
5) W. Strielkowski et al., "AI-Driven Adaptive Learning for Sustainable Educational Transformation," *Sustainable Development Journal*, 2025. [Online]. Available: https://www.scribd.com/document/901461552
6) J. R. Gómez Niño, L. P. Árias Delgado, A. Chiappe, and E. Ortega González, "Gamifying Learning with AI: A Pathway to 21st-Century Skills," *Computers Journal*, vol. 13, no. 12, pp. 324–338, 2024. [Online]. Available: https://www.mdpi.com/2073-431X/13/12/324
7) F. Weiß, "The Role of Gamification and Adaptive Learning in Engaging 21st-Century Students," *Digital First Magazine*, 2025. [Online]. Available: https://www.digitalfirstmagazine.com/the-role-of-gamification-and-adaptive-learning-in-engaging-21st-century-students
8) X. Wang, S. Wrede, L. van Rijn, and J. Wöhrle, "AI-Based Quiz System for Personalized Learning (iQS)," *DFKI Research Publications*, 2023. [Online]. Available: https://www.dfki.de/fileadmin/user_upload/import/14538_106756.pdf
9) R. Kumar, "Bridging the Gap: AI and the Future of Learning in India and Beyond," *Indian Education Review*, 2025. [Online]. Available: https://journal.esrgroups.org/jes/article/download/8690/5810/15789

10) L. Y. Tan, S. Hu, D. J. Yeo, and K. H. Cheong, "Artificial Intelligence-Enabled Adaptive Learning Platforms: A Review," *ResearchGate Preprint*, 2025. [Online]. Available: https://www.researchgate.net/publication/392257164

11) K. I. Vorobyeva, S. Belous, N. V. Savchenko, and S. P. Zhdanov, "Personalized Learning through AI: Pedagogical Approaches and Critical Insights," *Contemporary Educational Technology Journal*, 2025. [Online]. Available: https://www.cedtech.net/download/personalized-learning-through-ai-pedagogical-approaches-and-critical-insights-16108.pdf

12) V. A. Kusam, Z. Song, K. Kattan, and B. R. Maxim, "Evaluating the Effectiveness of Generative AI for Automated Quiz Creation: A Case Study," *ASEE Annual Conference & Exposition*, 2025. [Online]. Available: https://nemo.asee.org/public/conferences/365/papers/48701/view

13) I. Y. Zairon, T. S. M. Tengku Wook, S. M. Salleh, and H. A. Dahlan, "Adaptive Gamification in Collaborative Virtual Classrooms: A Systematic Review," *PeerJ Computer Science*, vol. 9, 2023. [Online]. Available: https://peerj.com/articles/cs-3146

14) E. du Plooy, D. Casteleijn, and D. Franzsen, "Personalized Adaptive Learning in Higher Education: A Scoping Review," *Open Ukrainian Citation Index*, 2024. [Online]. Available: https://ouci.dntb.gov.ua/en/works/9ZW5aO37

15) E. Ratinho and C. Martins, "The Role of Gamified Learning Strategies in Student Motivation: A Systematic Review," *Frontiers in Psychology*, 2023. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC10448467

16) A. I. Zourmpakis, K. Karpouzis, and K. Magoutis, "The Effects of Adaptive Gamification in Science Learning," *Computers Journal*, vol. 13, no. 12, 2024. [Online]. Available: https://www.mdpi.com/2073-431X/13/12/324

17) N. S. Farhah, A. Wadood, A. A. Alqarni, and M. I. Uddin, "Enhancing Adaptive Learning with Generative AI for Tailored Educational Support," *Journal of Disability Research*, 2025. [Online]. Available: https://scholar.google.com/citations?user=Khh5bXIAAAAJ

18) J. Jabbour, K. Kleinbard, O. Miller, R. Haussman, and V. Janapa Reddi, "SocratiQ: A Generative AI-Powered Learning Companion for Personalized Education," *Harvard University*, 2025. [Online]. Available: https://www.questionpro.com/blog/ai-quiz-generators

19) C. Isley et al., "Assessing the Quality of AI-Generated Exams: A Large-Scale Field Study," *arXiv preprint arXiv:2508.08314*, 2025. [Online]. Available: https://arxiv.org/abs/2508.08314

20) M. Srivastava and N. Goodman, “Question Generation for Adaptive Education,” *arXiv preprint arXiv:2106.04262*, 2021. [Online]. Available: https://arxiv.org/abs/2106.04262

21) M. Mittal and S. Sai, “A Comprehensive Review on Generative AI for Education,” *ScienceDirect Journal of AI in Education*, 2025. [Online]. Available: https://www.sciencedirect.com/article/pii/S2666920X25000694

22) S. Brown, “AI-Driven Assessment and Adaptive Feedback Systems in Modern Education,” *International Conference on AI in Learning*, 2024. [Online]. Available: https://arxiv.org/abs/2404.09000

23) L. Jones and P. Tran, “Gamified Learning Platforms: Motivation and Adaptive Feedback in Education,” *IEEE Access*, vol. 12, pp. 54320–54331, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10543522

24) Y. Li, “The Role of LLMs in Educational Content Generation,” *arXiv preprint arXiv:2309.14521*, 2023. [Online]. Available: https://arxiv.org/abs/2309.14521

25) D. K. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. ICLR*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980